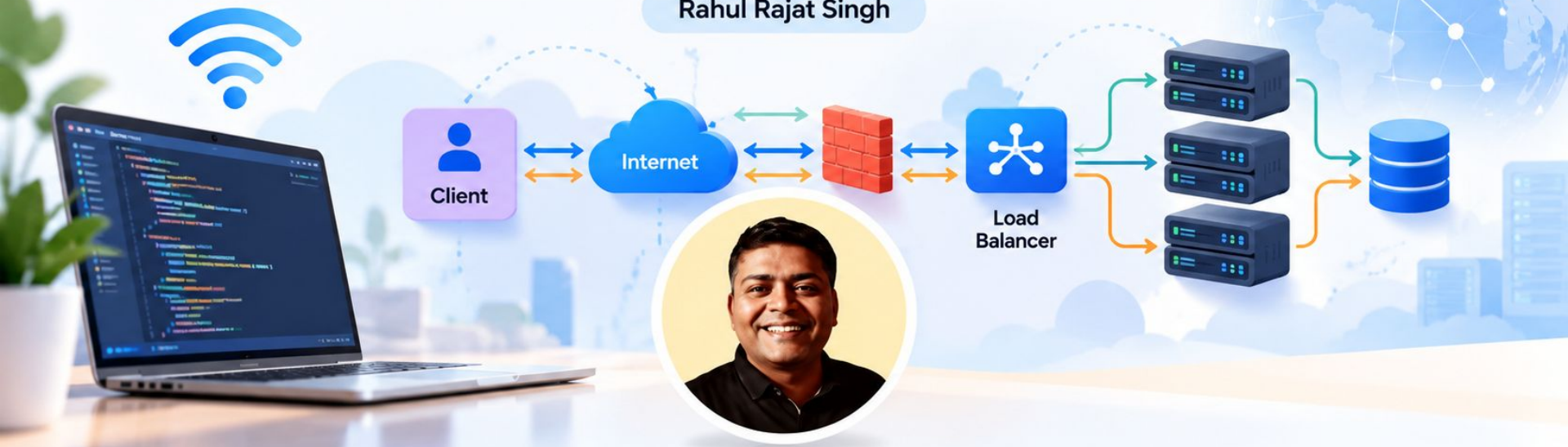
Introduction to Networking in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh



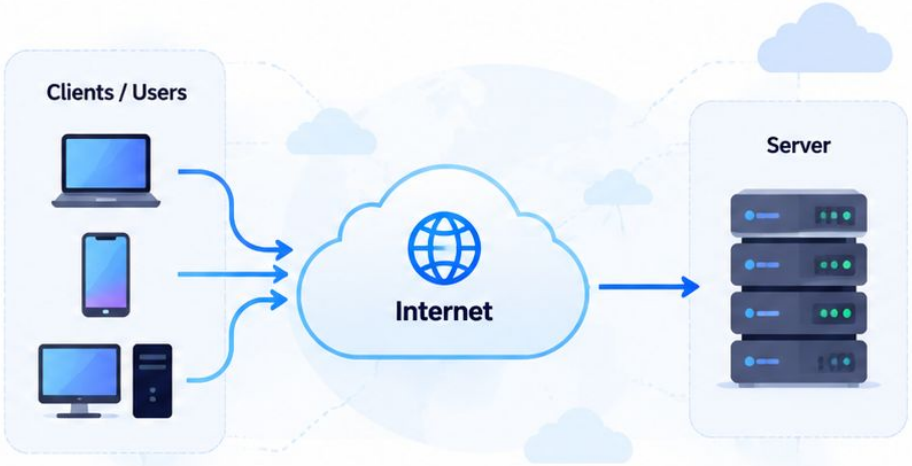
Instructor: Rahul Rajat Singh

Why Networking Matters in System Design?

Every system relies on data exchange between components.

Networking enables scalability, reliability, and performance.

Key areas where networking plays a crucial role:



Communication

Ensuring smooth data transfer between clients, servers, and databases.

Load Balancing

Distributing traffic efficiently to prevent overload on a single server.

Security

Protecting data from unauthorized access and cyber threats.

Efficiency

Optimizing network performance to reduce latency and improve user experience.

How Networking Impacts Large-Scale Systems

- Helps handle **millions of users** concurrently
- Enables fast and **efficient data exchange**
- Reduces latency & **improves system resilience**
- Essential for **cloud computing** & **distributed systems**



Instructor: Rahul Rajat Singh

Agenda

Key Networking Concepts in System Design



1 Understanding IP Addresses



2 How DNS Works



3 Client-Server Model Explained



4 Forward Proxy vs. Reverse Proxy



5 Introduction to Load Balancing



6 What is an API Gateway?



7 Content Delivery Networks (CDN) in System Design



Instructor: Rahul Rajat Singh

Wrap-Up & What's Next

1. **Networking is the foundation of system design**
   Everything in a distributed system communicates over a network.

2. **Next, we'll deep dive into networking components**
   Understand the building blocks that make communication possible.

3. **Coming up:**
   - IP Addressing & DNS



Next Lesson
IP Addressing & DNS

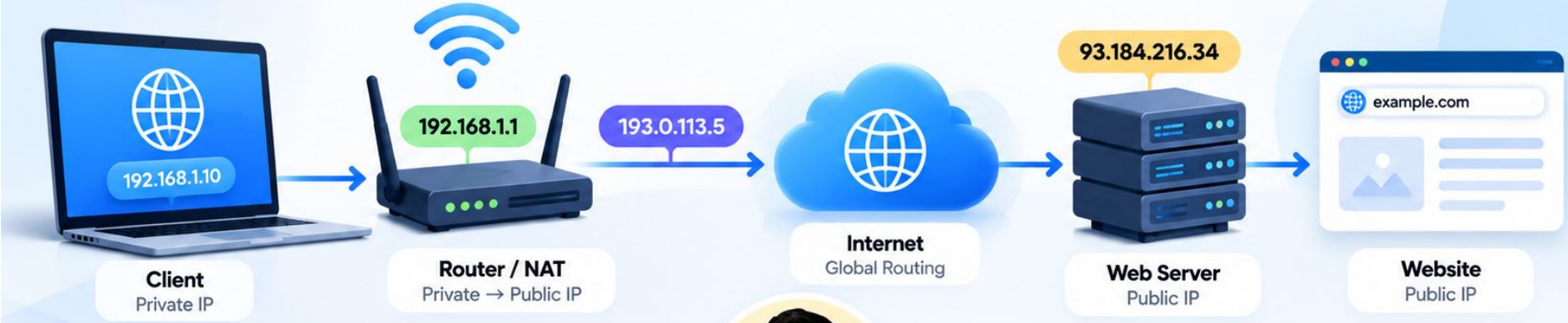


Instructor: Rahul Rajat Singh

Understanding IP Addresses in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews





Instructor: Rahul Rajat Singh

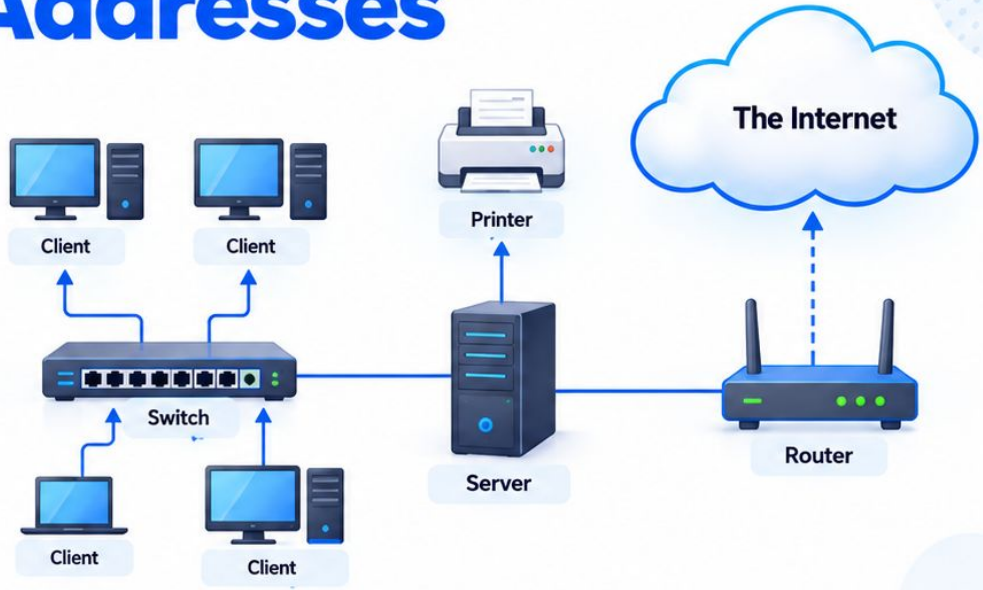
Introduction to IP Addresses

Unique Identifier

IP (Internet Protocol) addresses are unique numerical labels assigned to devices on a network.

Enables Communication

They enable communication between different machines, servers, and services across the internet.



Two Primary Versions

IPv4

- 32-bit address
- e.g. 192.168.1.1
- Widely used

IPv6

- 128-bit address
- e.g. 2001:db8::1
- Massive address space

Two Primary Categories

Public

- Globally unique
- Routable on the internet
- Used to access devices and services from anywhere

Private

- Used within internal networks
- Not directly accessible from the internet
- Common ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

Instructor: Rahul Rajat Singh

What is IPv4?

Definition
IPv4 (Internet Protocol Version 4) is the most widely used addressing system.

Address Format
0101
0101

32-bit address format (e.g., 192.168.1.1).

Total Addresses
~4.3 billion addresses ($2^{32} \approx 4,294,967,296$).

Common Uses
Traditional networking, web servers, and most current internet devices.

Challenges
Limited IPs, fragmentation, and security concerns.



Example IPv4 Address

192 . **168** . **1** . **1**

1st Octet	2nd Octet	3rd Octet	4th Octet
(0 – 255)	(0 – 255)	(0 – 255)	(0 – 255)

What is IPv6?

IPv6 (Internet Protocol Version 6)
is the next-generation IP standard.

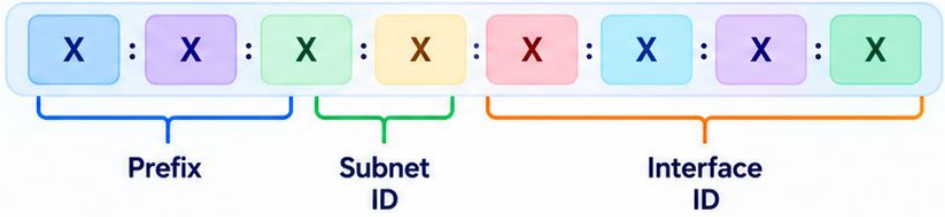
128-bit address format
Example:
`2001:0db8:85a3:0000:0000:8a2e:0370:7334`

Total addresses available
340 undecillion (virtually unlimited).

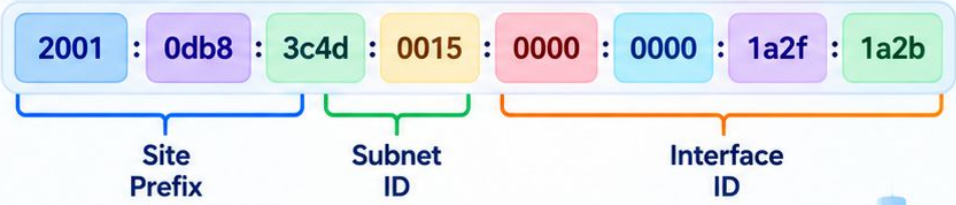
Designed for
IoT, mobile networks, and future scalability.

Key benefits
Larger address space, better security, and improved routing efficiency.

IPv6 Address Structure



Example:





Instructor: Rahul Rajat Singh

IPv4 vs. IPv6 – Key Differences



Feature	IPv4	IPv6
Address Size	32-bit	128-bit
Address Format	Decimal (e.g., 192.168.1.1)	Hexadecimal (e.g., 2001:db8::ff00:42:8329)
Address Space	~4.3 billion addresses	Virtually unlimited
Security	Relies on additional protocols	Built-in security (IPSec)
Performance	Limited due to NAT & fragmentation	More efficient routing & handling
Adoption	Still widely used	Slowly being adopted



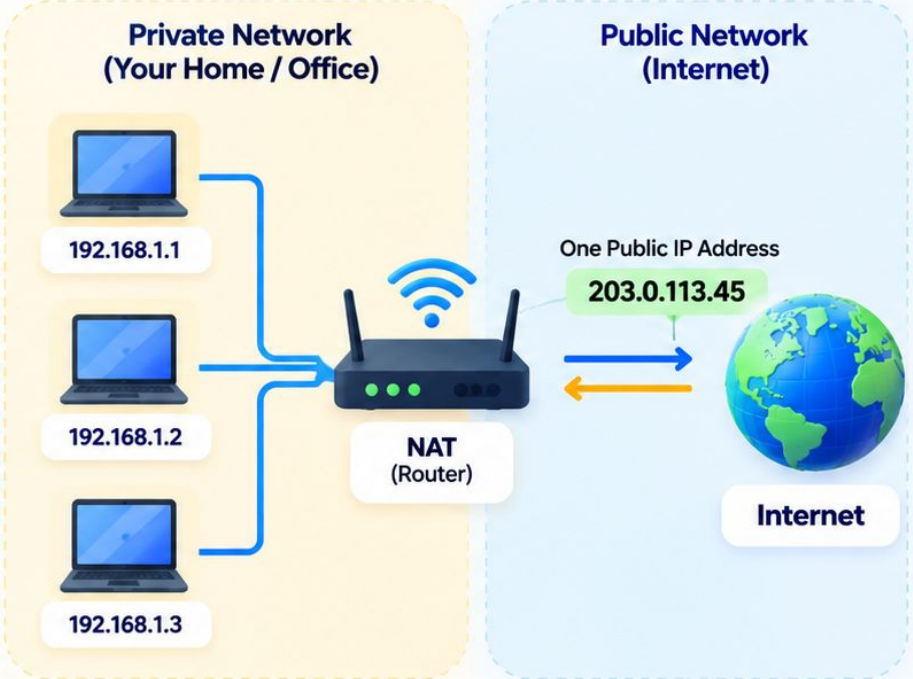
Private vs. Public IPs

Public IPs

- Assigned by ISPs (Internet Service Providers).
- Used to communicate over the internet.
- Unique worldwide.
- Example: 192.203.23.45

Private IPs

- Used within local networks (LANs, enterprises, homes).
- Cannot be accessed directly from the internet.
- Examples:
  - 10.0.0.0 - 10.255.255.255
  - 172.16.0.0 - 172.31.255.255
  - 192.168.0.0 - 192.168.255.255



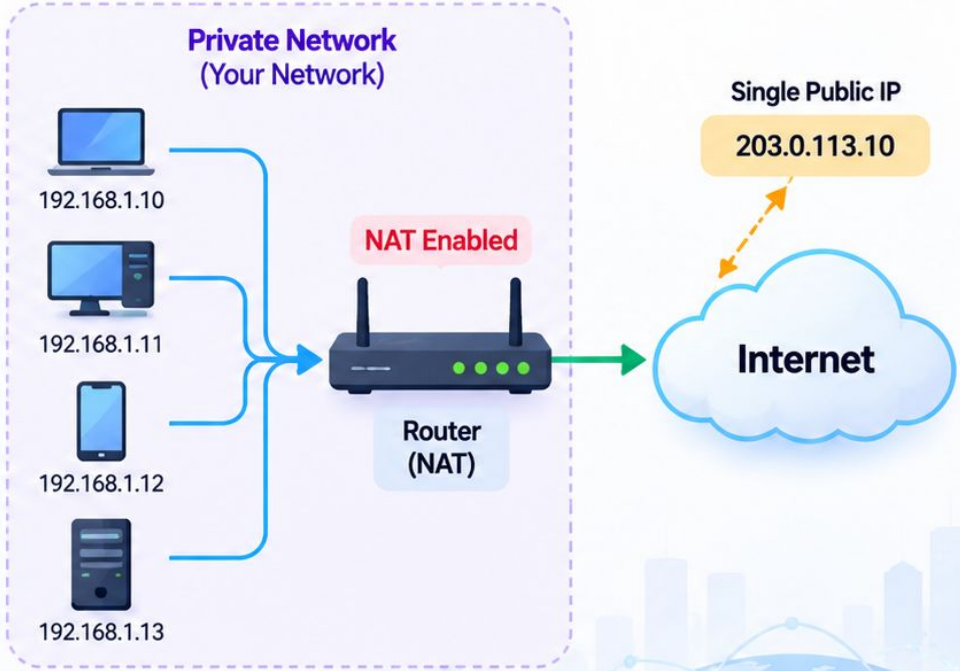
Why Do We Need Private IPs?

Conserves Public IP Addresses
Helps deal with IPv4 address limitation by allowing many devices to share a single public IP.

Enhances Security
Private IPs are not routable over the internet, keeping internal devices hidden from the public.

Enables Network Address Translation (NAT)
Allows multiple devices with private IPs to share a single public IP when accessing the internet.

Widely Used
Common in corporate networks, data centers, and cloud environments.



The Role of IPs in System Design

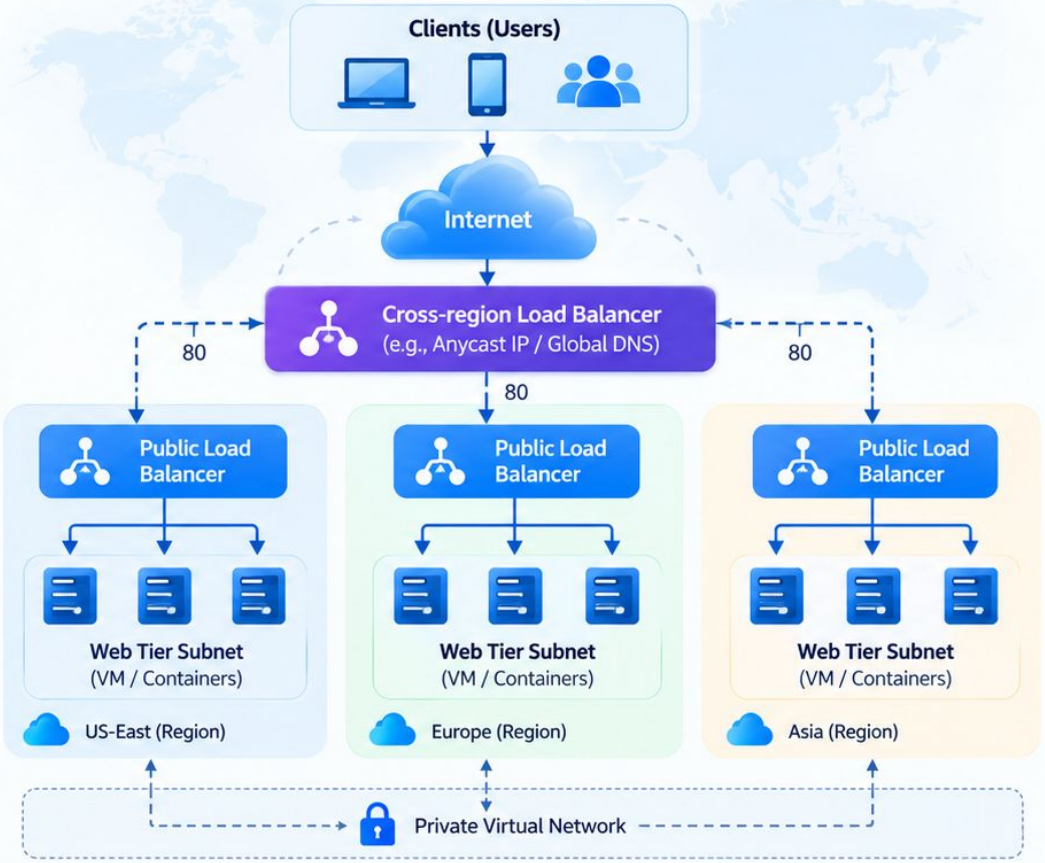
Scalability
Helps in designing distributed, multi-region, and cloud-based architectures.

Security
Enables firewall rules, VPNs, and private networking.

Load Balancing
Uses IP-based traffic distribution (e.g., Round-robin DNS, Anycast IPs).

Cloud Networking
Public, private, and hybrid cloud IP management (AWS, GCP, Azure).

Microservices & Containers
Use internal private IPs for communication.



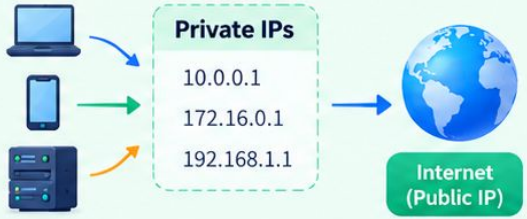
Common System Design Interview Questions on IPs



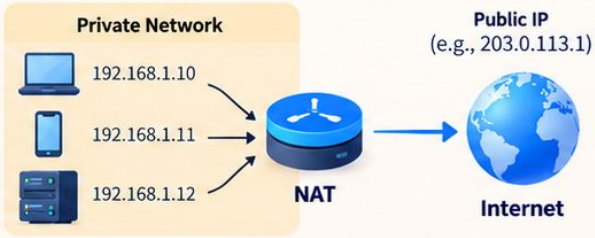
1 How do IPv4 and IPv6 addresses differ?

IPv4	IPv6
192.168.1.1	2001:0db8:85a3::7334
32-bit address (e.g., 192.168.1.1)	**128**-bit address (e.g., 2001:0db8:85a3::7334)

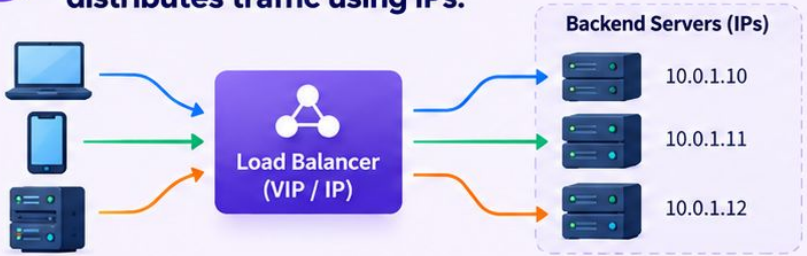
2 Why do we need private IPs in system design?



3 How does NAT help in addressing the IPv4 shortage?



4 Explain how a load balancer distributes traffic using IPs.



5 How does DNS resolve IP addresses in a large-scale system?



Instructor: Rahul Rajat Singh

Summary & What's Next

1. **IPv4 and IPv6 are the two primary internet protocols.**
   IPv4 is widely used, while IPv6 provides a much larger address space.

2. **Private vs. Public IPs help manage internal and external networking.**
   Private IPs are used inside networks, while public IPs are used to communicate over the internet.

3. **Networking & IPs are essential for system scalability, security, and performance.**
   They form the foundation of reliable and scalable systems.

4. **Coming up next:**
   - How DNS works and its role in system design.



NEXT LESSON



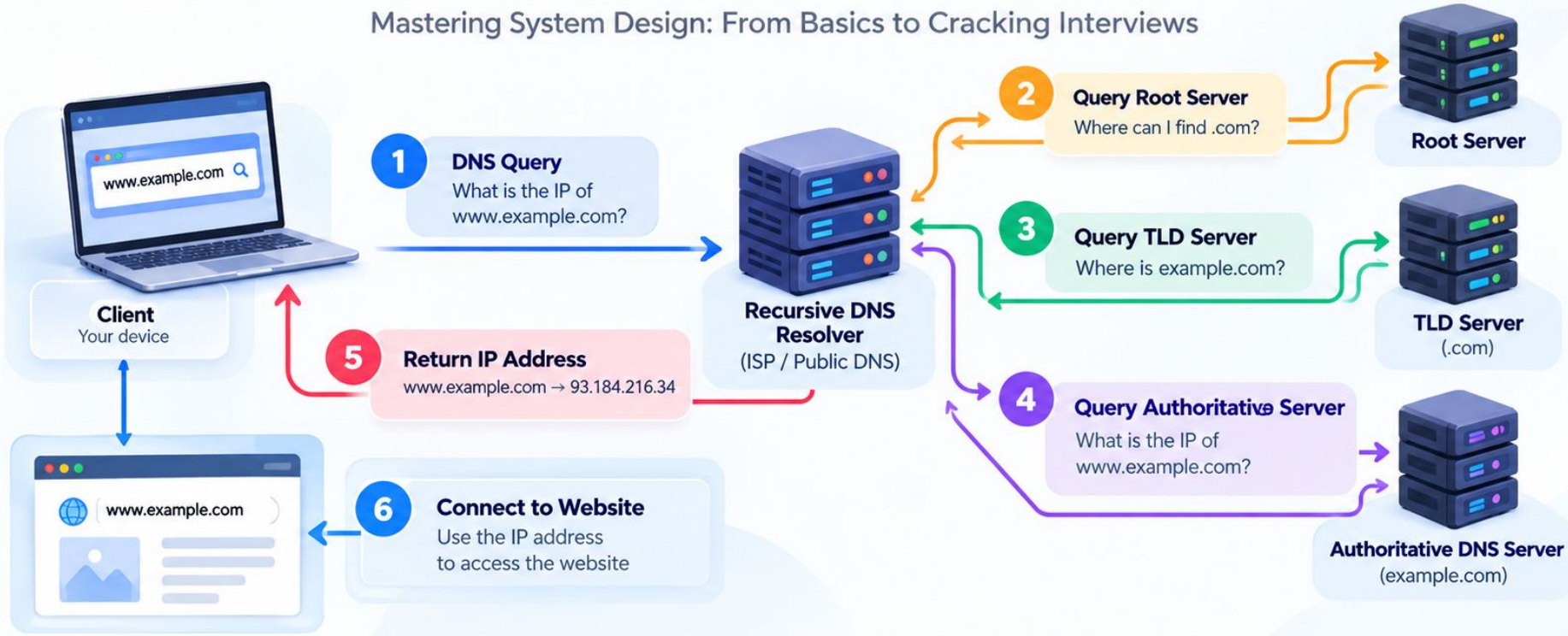
How DNS works and its role in system design.

Instructor: Rahul Rajat Singh

How DNS Works

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Introduction to DNS

What is DNS?

- The "phonebook of the internet."
- Translates human-readable domain names (e.g., google.com) into IP addresses.

Why is DNS Important?

- Enables accessibility of websites.
- Supports distributed, scalable internet architecture.



Instructor: Rahul Rajat Singh

Types of DNS Servers

1 

Root Name Servers

Top-level servers handling .com, .org, etc.

2 

TLD Name Servers

Specific to domains like .com, .net, etc.

3 

Authoritative Name Servers

Store domain-specific records (e.g., IP addresses).

4 

Recursive Resolvers

Handle queries on behalf of users, contacting multiple DNS servers if necessary.



User Request
www.example.com

4 

Recursive Resolver
(ISP or Public DNS)

Returns IP address

1 

Root Name Servers
(Top-Level Servers for .com, .org, etc.)

Refers to TLD servers

2 

TLD Name Servers
(Handles .com, .net, etc.)

Refers to authoritative servers

3 

Authoritative Name Servers
(Stores Domain-Specific Records)



Resolved IP Address Returned
93.184.216.34



Website Loads in Browser
www.example.com

Instructor: Rahul Rajat Singh

DNS Caching and Performance Optimization



1 Why caching matters?

- Reduces latency.
- Reduces load on DNS servers.

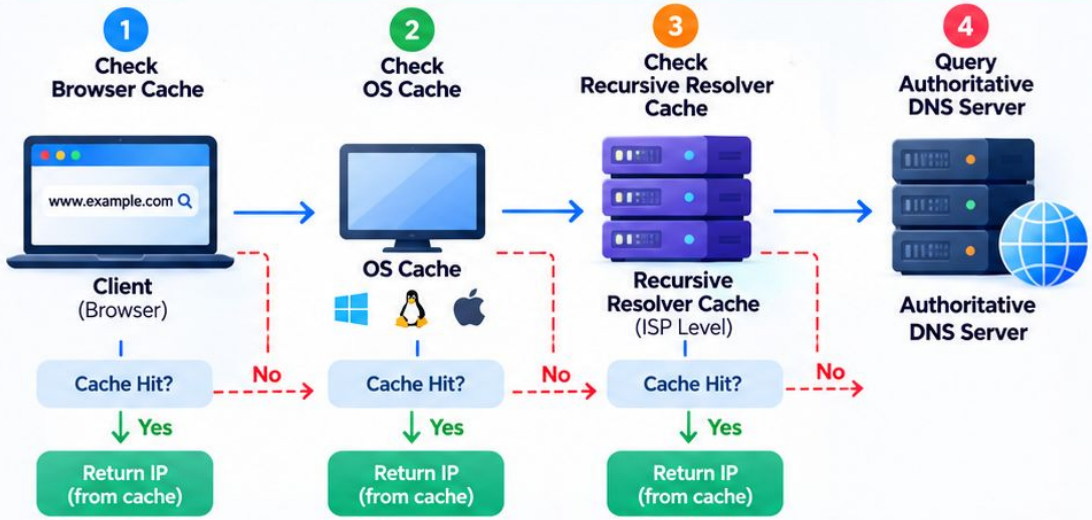
2 Where caching occurs?

- **Browser cache**
  Stores previous lookups.
- **OS cache**
  (etc/hosts, Windows DNS cache).
- **Recursive Resolver cache**
  (ISP level).

3 Time-To-Live (TTL)

- Determines how long a cached record is valid.

How DNS Caching Works



TTL (Time-To-Live)
Determines how long a cached record is valid.

Example:

```
example.com IN A 93.184.216.34
```

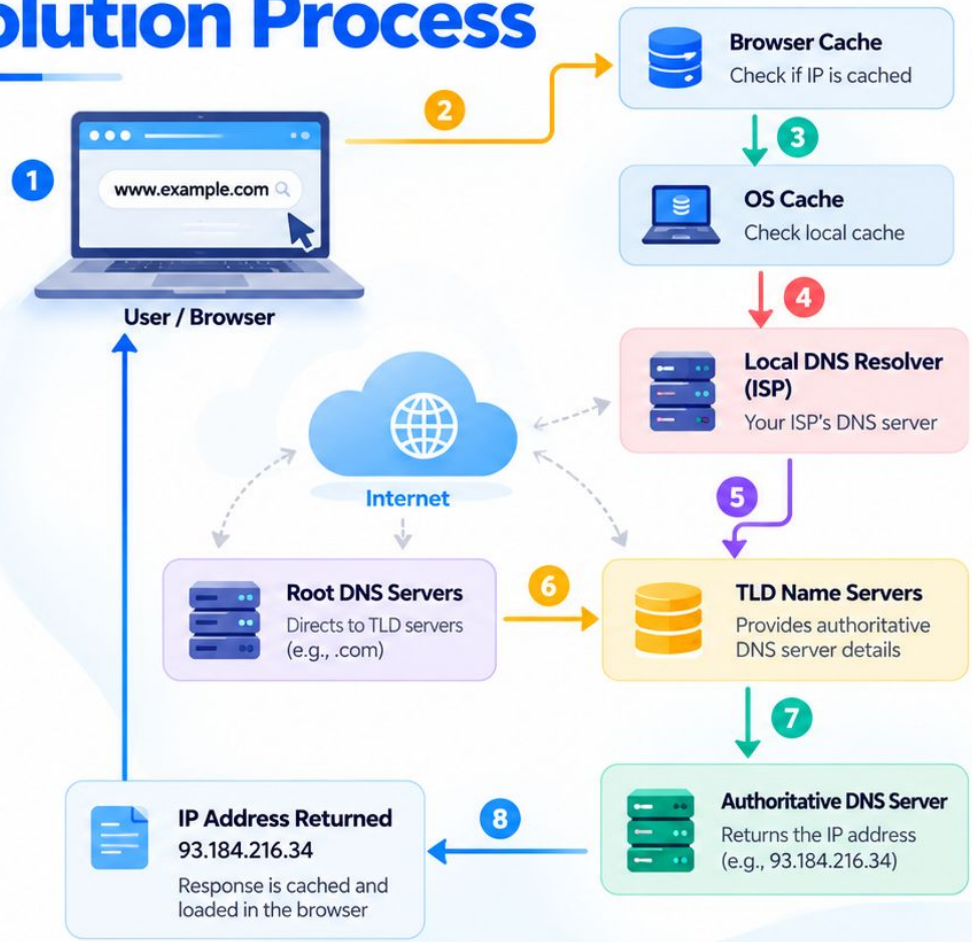
TTL = 300s

The record will be cached for 300 seconds (5 minutes).

Instructor: Rahul Rajat Singh

The Domain Name Resolution Process

1. **User types a domain**
   You enter a domain name in the browser (e.g., www.example.com)
2. **Browser cache check**
   Browser checks if the IP is already cached.
3. **OS cache check**
   Operating System (OS) checks its local cache.
4. **Query to local DNS resolver (ISP)**
   If not found, query goes to the configured DNS server (usually by your ISP).
5. **Query to Root DNS servers**
   If still not found, the resolver queries a Root DNS server.
6. **TLD name servers**
   Root server refers to the TLD (e.g., .com) name servers, which provide the authoritative server details.
7. **Authoritative DNS server**
   The authoritative DNS server for the domain returns the IP address.
8. **IP returned & cached**
   The IP address is returned to the browser and cached for future use.



Instructor: Rahul Rajat Singh

Importance of DNS in Large-Scale Systems

1 Ensuring High Availability

- Load balancing using DNS.
- Anycast DNS for faster global resolution.

2 DNS Failover Strategies

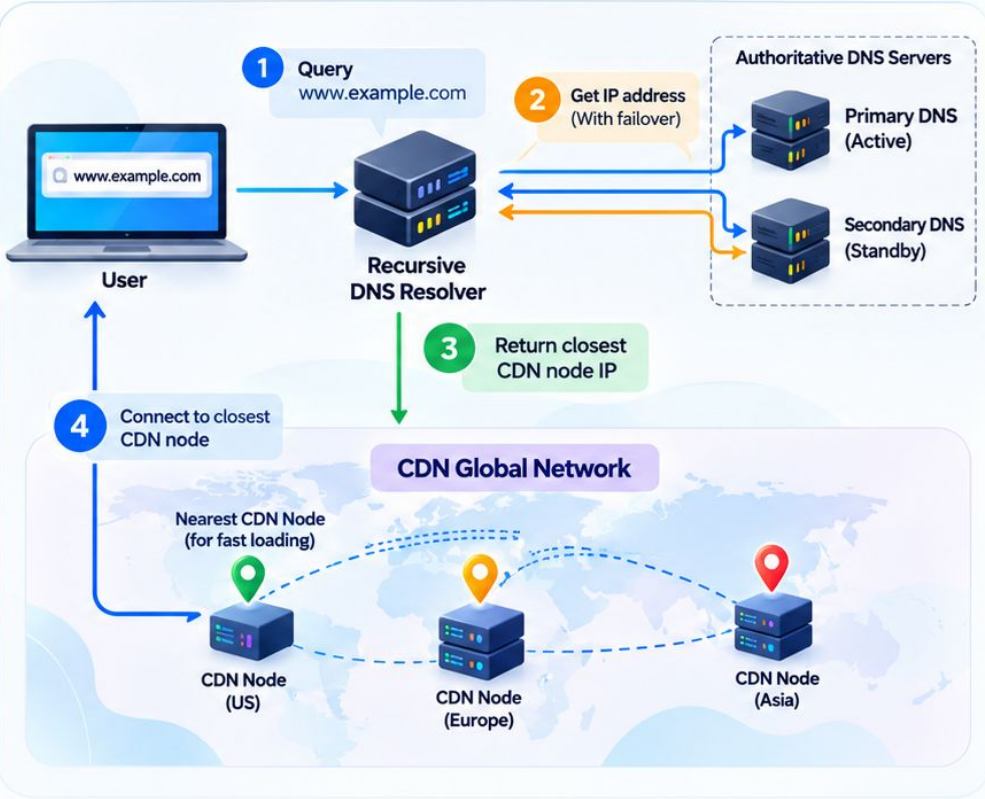
- Primary and secondary DNS servers.
- Automatically route traffic during failures.

3 Content Delivery Networks (CDNs) & DNS

- How DNS directs users to the nearest CDN node for fast loading.
- Improves performance and reduces latency.

4 DNS Security Risks

- DNS poisoning, cache poisoning, and DDoS attacks.
- Use DNSSEC, rate limiting, and monitoring.



Instructor: Rahul Rajat Singh

Interview Questions Related to DNS

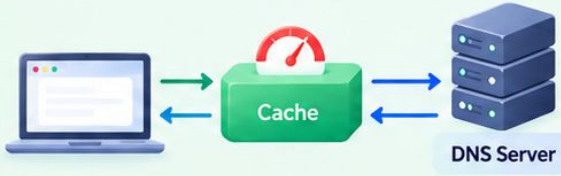
1 Explain the DNS resolution process step by step.



2 What is the difference between authoritative and recursive DNS servers?



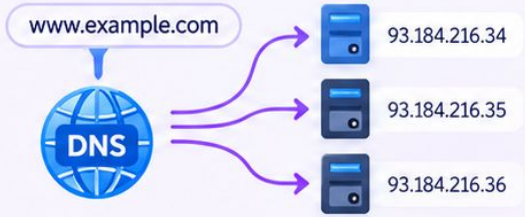
3 How does DNS caching improve performance? Where does it occur?



4 What is TTL in DNS, and why is it important?



5 How does DNS-based load balancing work in large-scale systems?



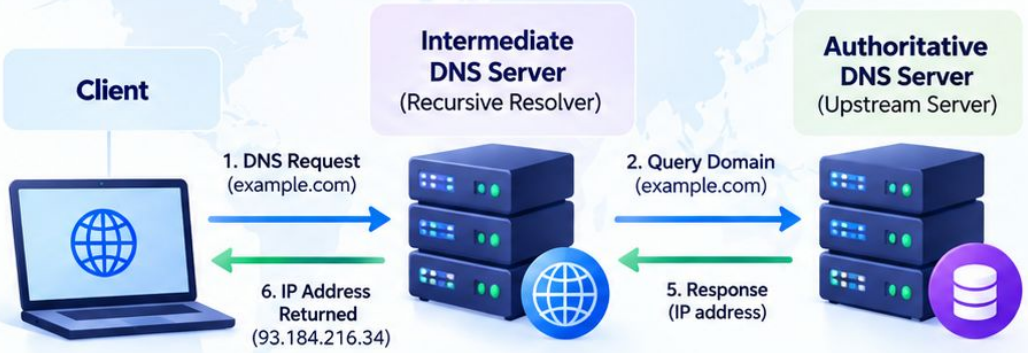
6 What are common DNS-related security threats, and how can they be mitigated?



Instructor: Rahul Rajat Singh

Summary & Takeaways

1. **DNS is critical** for internet functionality.
2. **Caching improves** efficiency and reduces latency.
3. **DNS load balancing & failover** are essential for large-scale systems
4. **Security threats exist,** but best practices help mitigate them



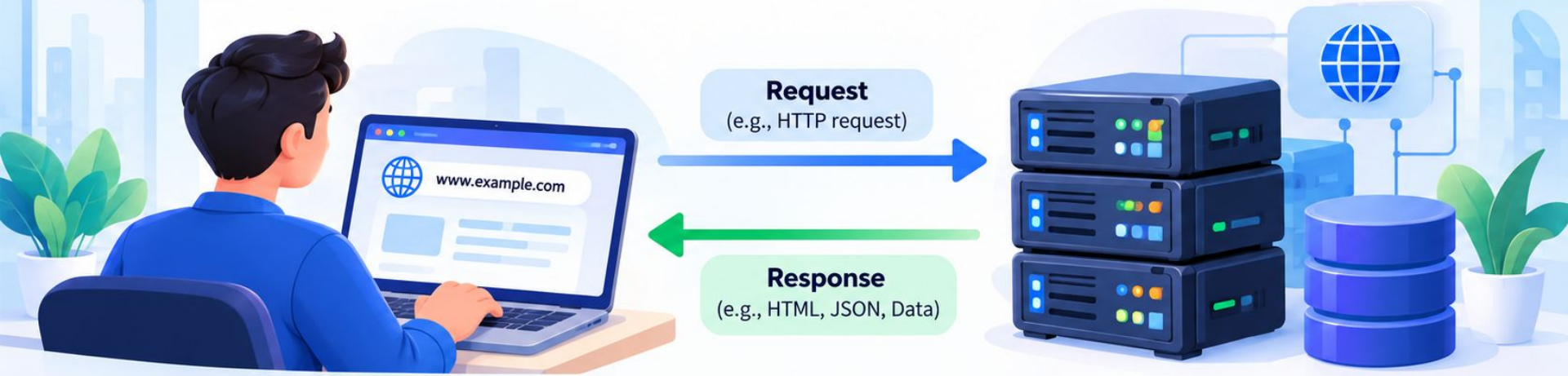
What's Next?

- Client-Server Model Explained

Client-Server Model Explained

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

What is the Client-Server Model?

1

Definition

A computing model where clients request services and servers provide them.

2

Foundation

Foundation of modern web, database, and application architectures.

3

Why is it important?

- Enables efficient resource management.
- Used in web browsing, email, APIs, databases, and cloud services.

4

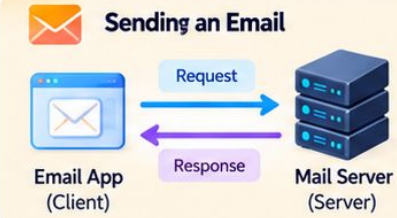
Real-world examples

- Browsing a website (Browser = Client, Web Server = Server).
- Sending an email (Email App = Client, Mail Server = Server).



Real-World Examples





Instructor: Rahul Rajat Singh

Key Components of the Client-Server Model

1 Client



- The user-facing application that sends requests.
- Examples: Web browser, mobile app, API consumer.

2 Server

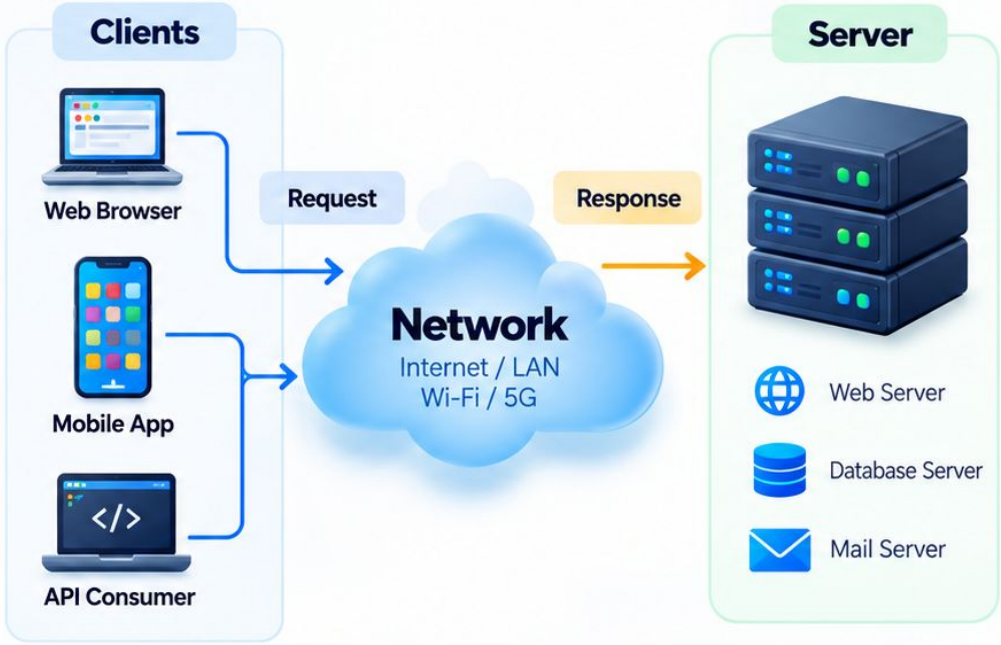


- A system that processes requests and returns responses.
- Examples: Web server, database server, mail server.

3 Network



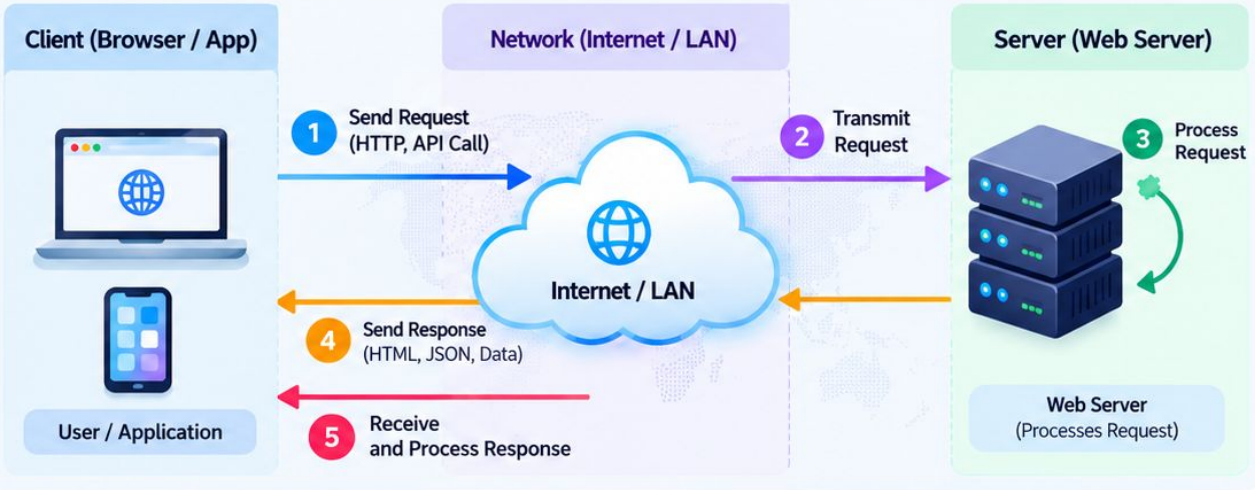
- The medium that facilitates communication between clients and servers.
- Examples: Internet, LAN, Wi-Fi, 5G.



How Clients and Servers Communicate

Basic Steps

1. Client sends a request (HTTP, SQL query, etc.).
2. Request is transmitted over the network.
3. Server receives and processes the request.
4. Server sends a response.
5. Client receives and processes the response.



Types of Communication

Request/Response Model

(e.g., HTTP, REST APIs)

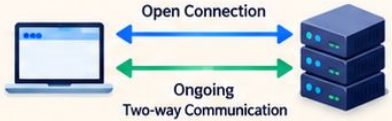
- Client sends a request, server responds.
- Simple and widely used.
- Common for web applications and APIs.



Persistent Connections

(e.g., WebSockets, FTP sessions)

- Keeps the connection open.
- Allows two-way communication.
- Used for real-time applications like chat, live updates, and file transfers.



Instructor: Rahul Rajat Singh

The Request-Response Cycle (HTTP Example)

1. **Browser request**
   Browser requests https://example.com.



2. **DNS resolution**
   DNS resolves domain to IP address.



3. **Send HTTP request**
   Browser sends an HTTP GET request to the web server.



4. **Server processes request**
   Web server processes request, fetches HTML, queries the database if needed.

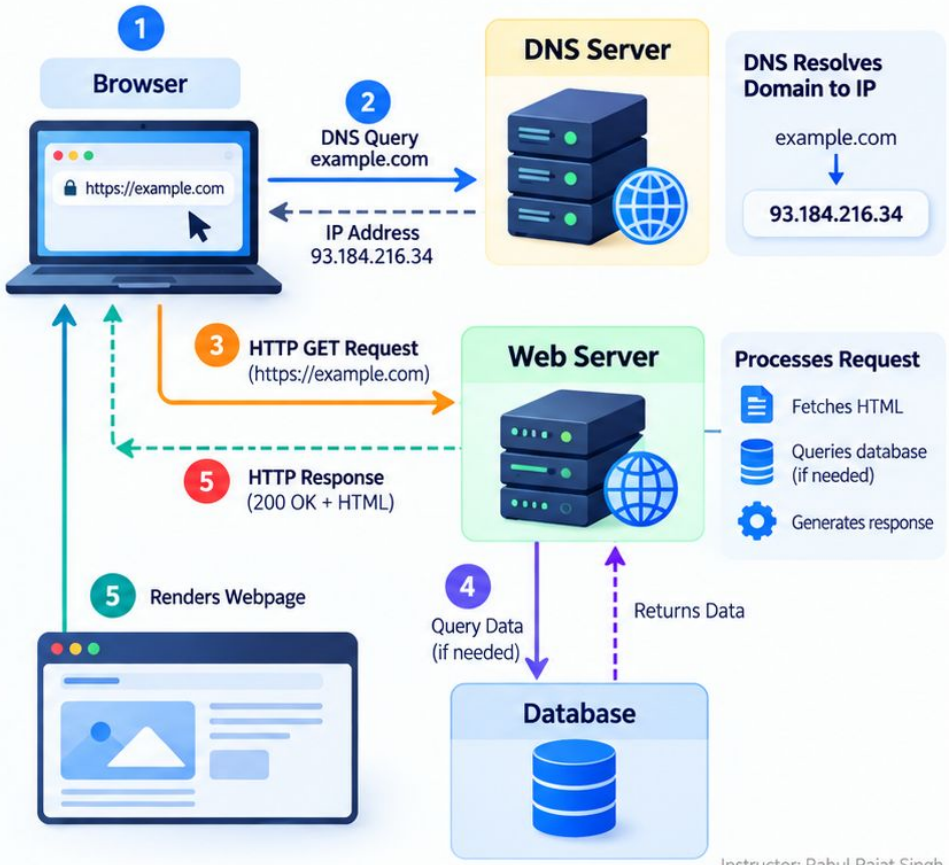


5. **Send HTTP response**
   Server sends an HTTP response with a status code (200 OK) and the webpage content.



6. **Browser renders webpage**
   Browser renders the webpage.



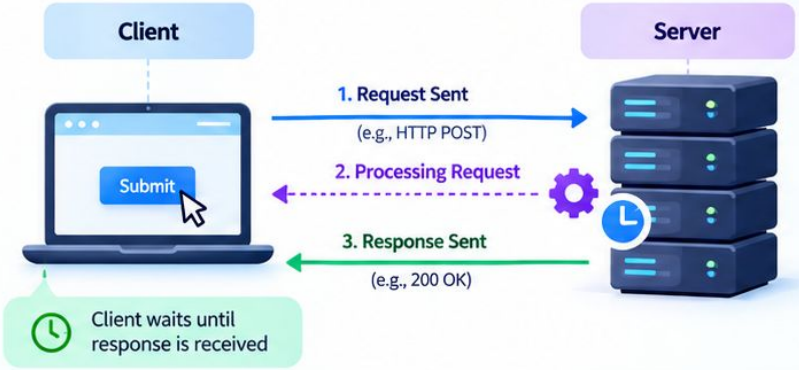


Instructor: Rahul Rajat Singh

Synchronous vs Asynchronous Communication

Synchronous Communication

- Client waits for a response before proceeding.
- Example: A user submits a form and waits for a confirmation message.
- Used in REST APIs, traditional web apps.

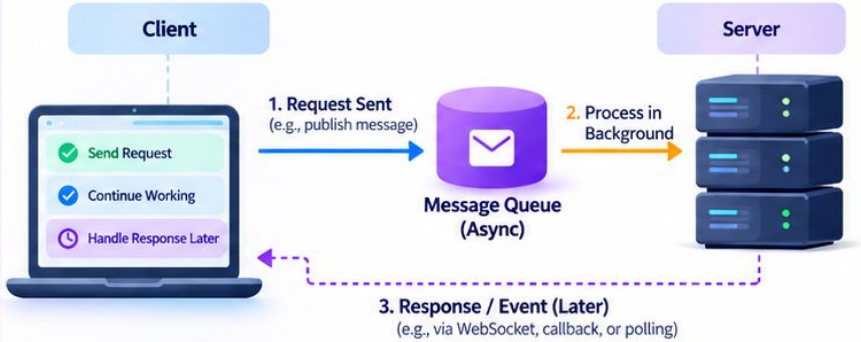


Real-World Examples

- REST APIs
- Traditional Web Apps
- Form Submissions

Asynchronous Communication

- Client doesn't wait for a response and can perform other tasks.
- Example: WebSockets for real-time chat, AJAX for background requests.
- Used in messaging apps, live updates, IoT applications.



Real-World Examples

- Messaging Apps (e.g., WhatsApp, Slack)
- Live Updates (e.g., notifications, real-time feeds)
- IoT Applications (e.g., sensor data)

Stateless vs Stateful Servers

1 Stateless Servers



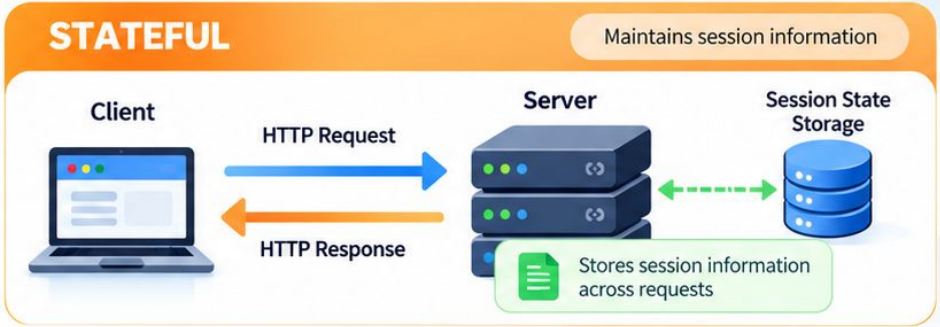
- **No memory** of past interactions; each request is independent.
- **Example:** REST APIs, HTTP servers.
- **Benefits:** Scalability, easy caching, load balancing.

2 Stateful Servers



- **Maintain** session information across requests.
- **Example:** WebSockets, multiplayer games, banking applications.
- **Benefits:** Personalization, seamless user experiences.





Common Stateless Examples

- REST APIs
- HTTP Servers
- Microservices
- CDN (Static Content)

Common Stateful Examples

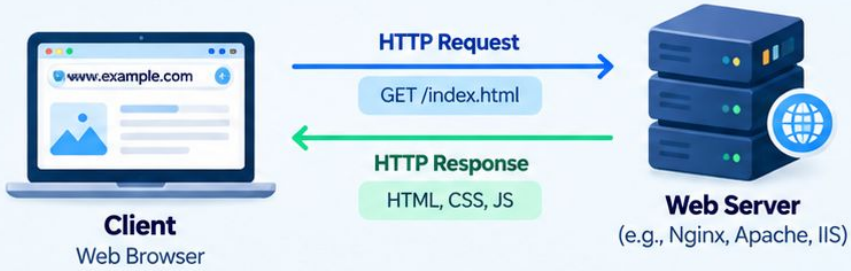
- WebSockets (Real-time apps)
- Multiplayer Games
- Banking Applications
- Personalized User Experiences

Instructor: Rahul Rajat Singh

Real-World Examples

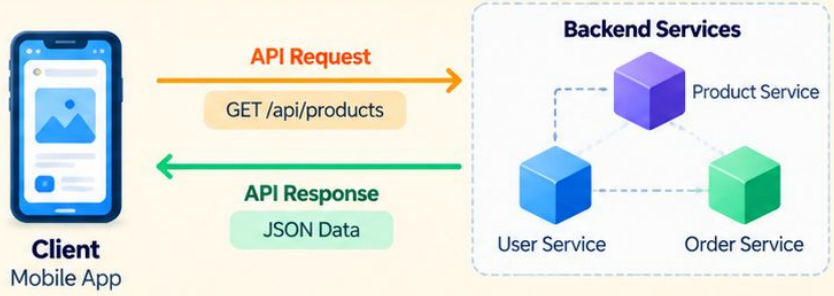
1 Web Applications

Browsing a website (Client = Browser, Server = Web Server).



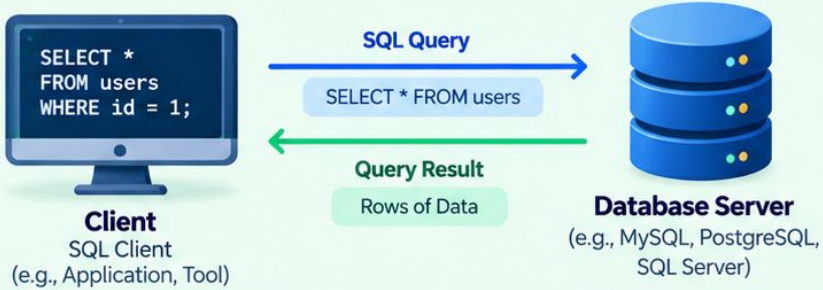
2 APIs & Microservices

Mobile apps calling backend APIs (REST, GraphQL).



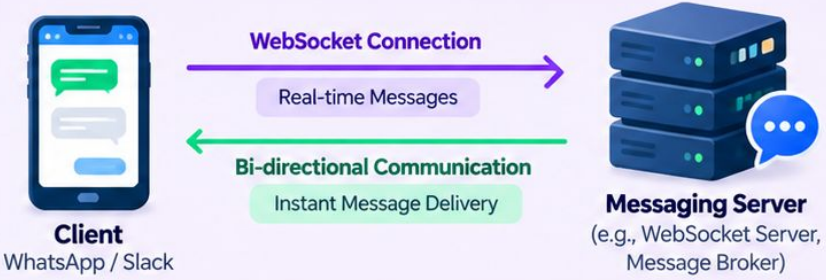
3 Databases

A client querying a database (SQL Client → Database Server).



4 Messaging Systems

WhatsApp, Slack using WebSockets for real-time communication.



Interview Questions on the Client-Server Model

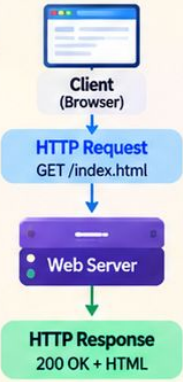
1 Fundamental Questions

1. What is the client-server model, and how does it work?
2. How does a client communicate with a server?
3. What are some real-world examples of the client-server model?
4. What is the difference between a client and a server?



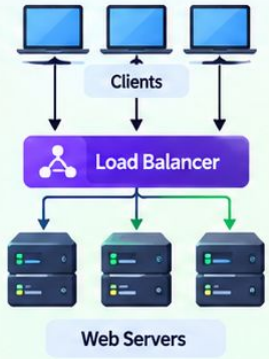
2 Request-Response Cycle Questions

1. Explain the HTTP request-response cycle with an example.
2. What are the key differences between synchronous and asynchronous communication?
3. How does a browser load a webpage? Walk me through the steps.
4. What happens in the network, server, and client during a request-response cycle?



3 Architecture and Design Questions

1. What is the difference between stateless and stateful servers?
2. How does caching improve performance in a client-server model?
3. How do load balancers work in a client-server architecture?



4 Advanced Questions

1. What are some security challenges in the client-server model?
2. How does WebSockets differ from traditional request-response communication?
3. What are some limitations of the client-server model? How can they be addressed?
4. How would you design a scalable client-server system for a high-traffic application?

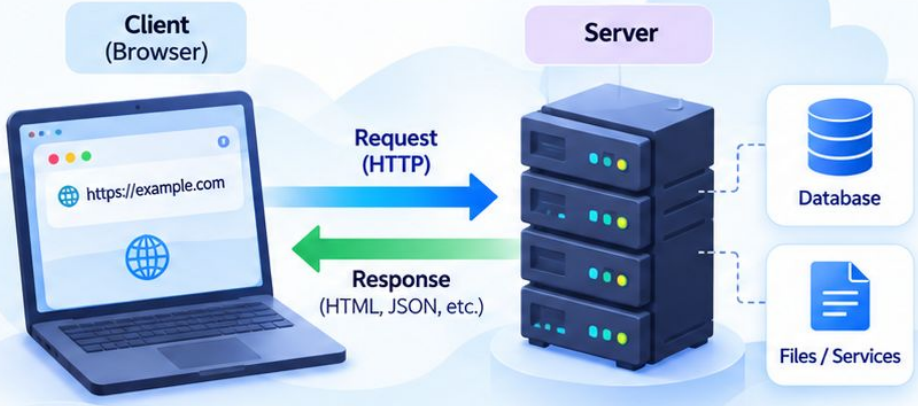


Instructor: Rahul Rajat Singh

Wrap up and What's Next...

Key Takeaways

1. **The client-server model**
   underpins most modern computing.
2. **Different communication patterns**
   (sync vs async, stateless vs stateful)
   optimize performance for different use cases.
3. **Understanding this model**
   helps in system design and scalability planning.



Next Steps

Forward Proxy vs. Reverse Proxy

In the Next Lecture...

We'll dive into Forward Proxy vs. Reverse Proxy in detail and see how they are used in real-world systems.

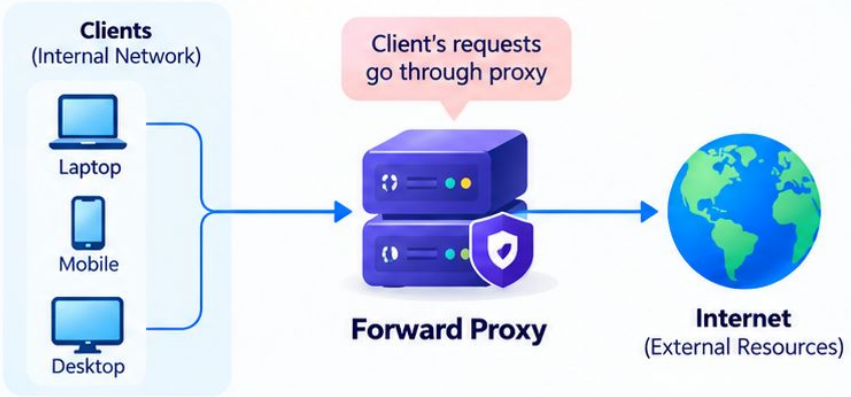
Instructor: Rahul Rajat Singh

Forward Proxy vs. Reverse Proxy

Networking & Communication (System Design Fundamentals)

Forward Proxy

Acts on behalf of the client to access external resources.



Use Cases

- Access control
- Content filtering
- Anonymity

Examples

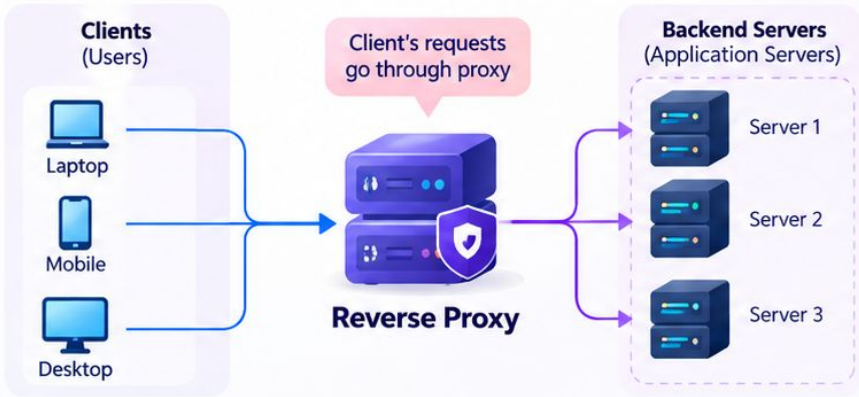
- Corp. proxy for internet access
- VPN, Web proxy

Benefits

- Hides client identity
- Enforces policies
- Controls outbound access

Reverse Proxy

Acts on behalf of the server to handle client requests.



Use Cases

- Load balancing
- SSL/TLS termination
- Security (WAF)
- Routing to backend services

Examples

- Nginx, Apache
- AWS ALB / Azure Application Gateway
- Cloudflare

Benefits

- Improves security
- Distributes traffic
- Hides backend servers
- Enables scalability and high availability

Instructor: Rahul Rajat Singh

What is a Proxy?

1. A proxy is an **intermediary server** between a client and another server.
2. Proxies help with **security, caching, traffic control, and anonymity.**
3. Two main types: **Forward Proxy & Reverse Proxy.**



1 Forward Proxy

Acts on behalf of **clients**, accessing external servers.



- Hides client's IP
- Controls access
- Caches content
- Improves security
- Useful in corporate networks

2 Reverse Proxy

Acts on behalf of **servers**, handling client requests.



- Hides backend servers
- Load balancing
- SSL termination
- Caching
- Improves security
- Used in modern web architectures

Instructor: Rahul Rajat Singh

What is a Forward Proxy?

- Sits between a client and the internet (used by clients).
- Clients connect to the forward proxy instead of directly accessing websites.
- Can filter content, provide anonymity and cache requests.



Common Use Cases

Bypassing geo-restrictions
Access websites that are blocked in your region.

Anonymous browsing (VPNs, TOR)
Hides your real IP address.

Caching web pages
Stores frequently accessed content for faster access.

How it Helps

Filter Content
Block unwanted websites or content.

Provide Anonymity
Hide the client's real IP address.

Cache Responses
Store frequently accessed pages.

Enforce Policies
Control and monitor internet usage in organizations.

Instructor: Rahul Rajat Singh

What is a Reverse Proxy?

- Sits between users and backend servers (used by servers).
- Clients don't directly communicate with backend servers—requests go through the reverse proxy.
- Used for load balancing, caching, security, and SSL termination.



Common Use Cases

Load Balancing
Distribute traffic across multiple servers.

Caching Content
Cache static content to improve speed.

DDoS Protection & Security
Protect backend servers from attacks and enhance security.

Instructor: Rahul Rajat Singh

Key Differences — Forward vs. Reverse Proxy

Forward Proxy



Position	Between client & internet
Who Uses It?	Clients (users, browsers, apps)
Purpose	Anonymity, filtering, access control
Example	VPN, Web Proxy

VS

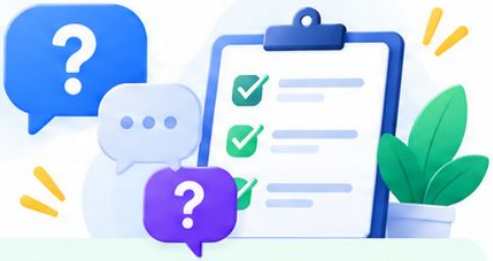
Reverse Proxy



Position	Between users & backend servers
Who Uses It?	Web servers, application servers
Purpose	Load balancing, caching, security
Example	Nginx, Cloudflare, AWS ALB

Instructor: Rahul Rajat Singh

Interview Questions on Forward Proxy vs. Reverse Proxy



1 Basic Concepts



1. What is a proxy server, and why is it used?
2. Explain the key differences between a forward proxy and a reverse proxy.

2 Technical Deep Dive

1. How does a forward proxy improve security and privacy?
2. How does a reverse proxy help in load balancing and caching?
3. What are some real-world examples of forward and reverse proxies?



3 Implementation & Use Cases

1. When should you use a forward proxy vs. a reverse proxy in system architecture?
2. What are some common tools and technologies used for each type of proxy?
3. How does a reverse proxy protect backend servers from DDoS attacks?



4 Advanced Topics

1. How does SSL termination work in a reverse proxy?
2. What are the advantages of using Cloudflare, Nginx, or HAProxy as a reverse proxy?
3. How does a transparent proxy differ from a forward or reverse proxy?



Instructor: Rahul Rajat Singh

Wrap-Up & What's Next?

Key Takeaways

1. A **forward proxy** helps clients interact with the internet securely.
2. A **reverse proxy** protects and optimizes backend servers.
3. Understanding proxy usage helps in designing scalable & secure architectures.

Forward Proxy — Helps clients access the internet securely



Reverse Proxy — Protects and optimizes backend servers



Next Topic:

Load Balancers

- How load balancers distribute traffic
- L4 vs L7 load balancing
- Real-world use cases and architectures

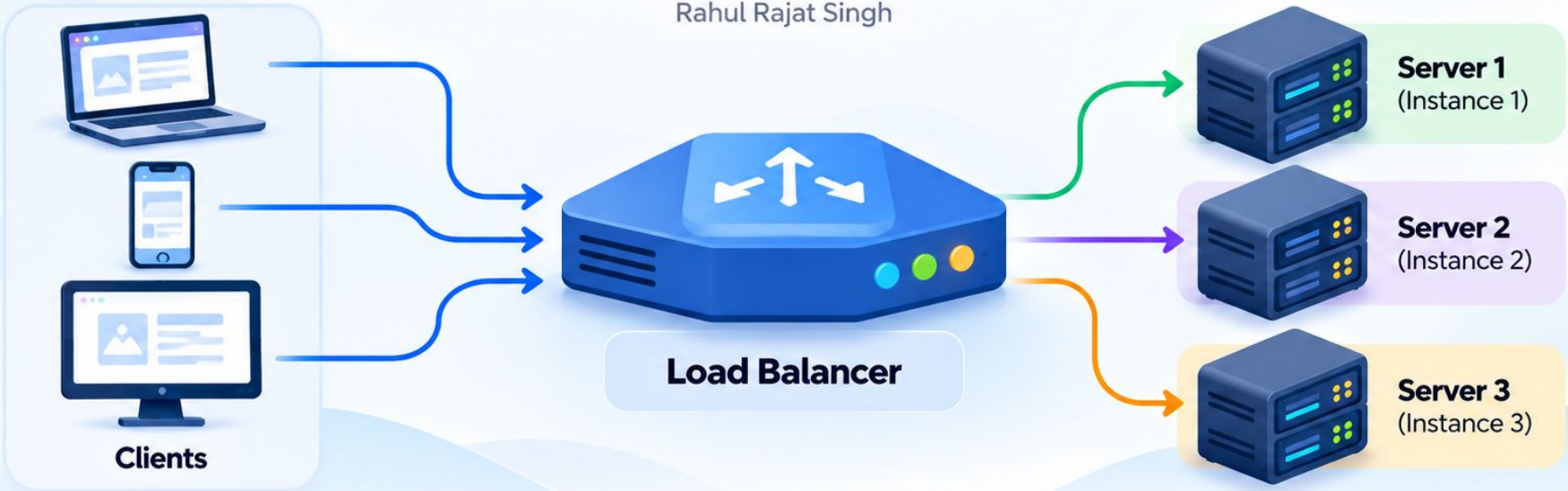
Instructor: Rahul Rajat Singh

Introduction to Load Balancing

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh



Introduction to Load Balancing

Modern applications must handle **growing traffic.**

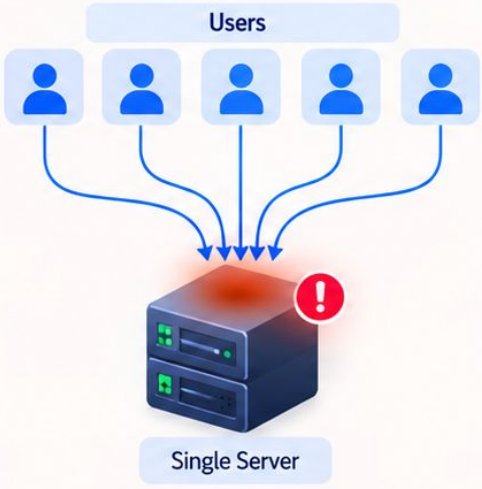
A single server eventually becomes a **bottleneck.**

Load balancing enables systems to **scale beyond one machine.**

It is a foundational building block of **distributed systems.**

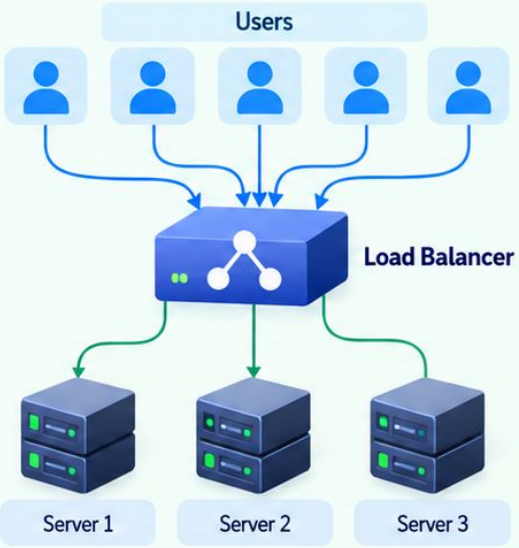
Without Load Balancing

A single server becomes a bottleneck



With Load Balancing

Distributes traffic across multiple servers



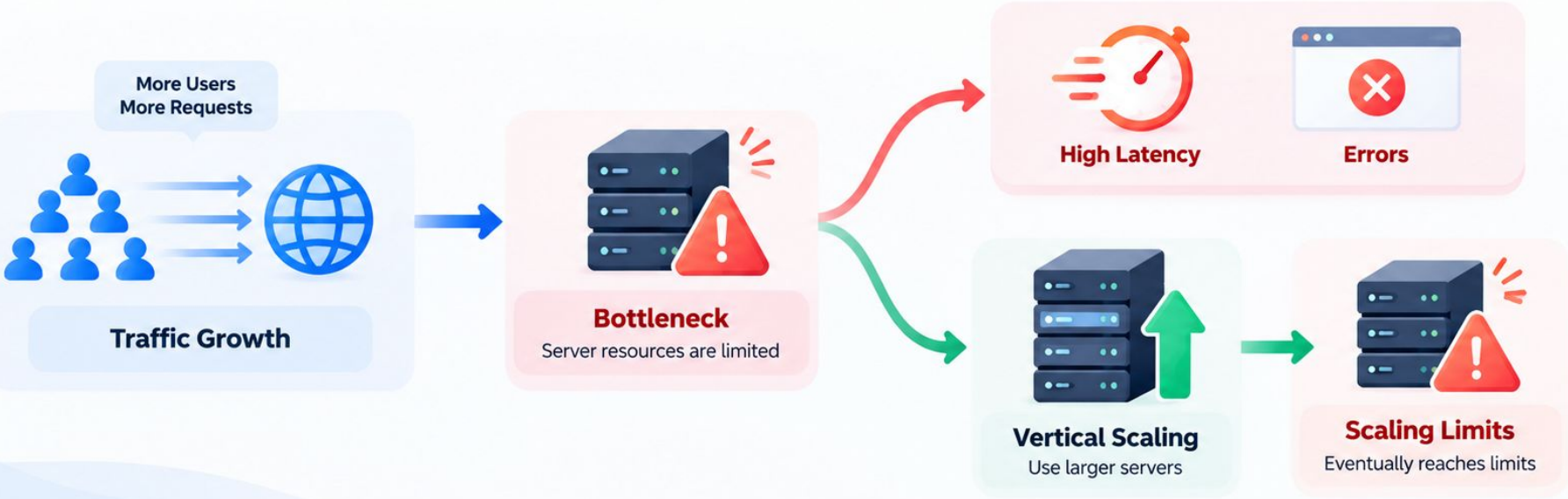
The Scaling Challenge

Server resources are **finite.**

Growing traffic increases **latency and failures.**

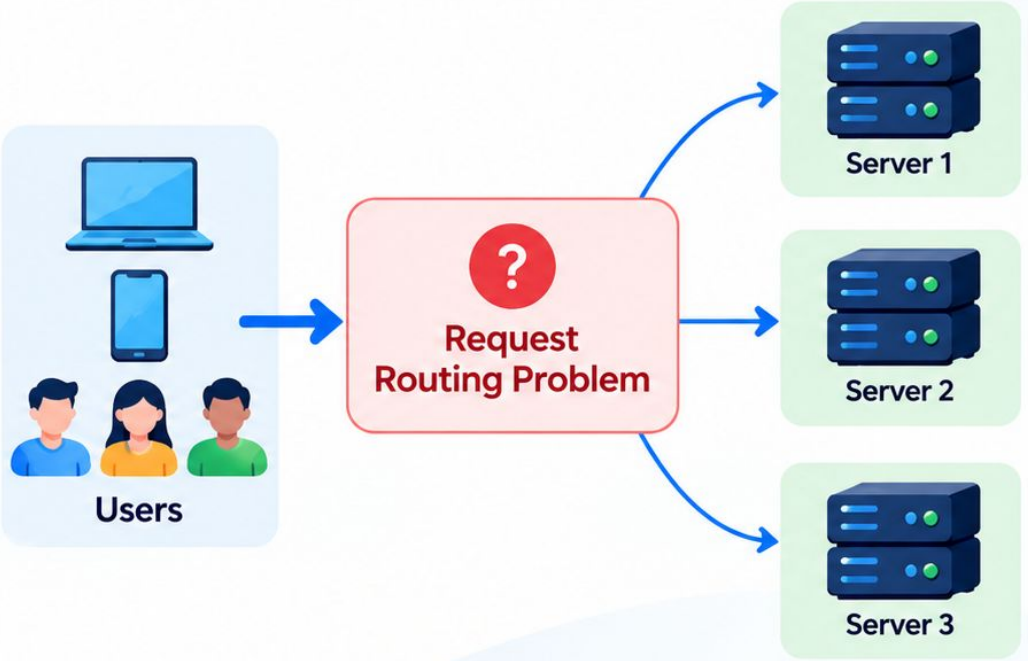
Larger servers help only **temporarily.**

Vertical scaling eventually **reaches limits.**



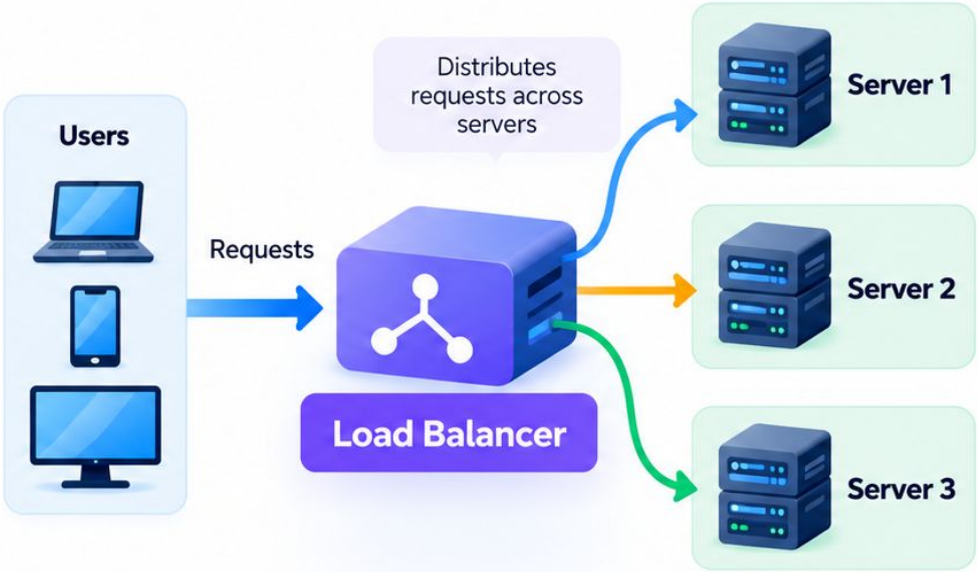
Horizontal Scaling Creates a New Problem

1. Capacity grows by adding servers.
2. Workload can be shared across machines.
3. Users should not choose servers manually.
4. Traffic now needs a distribution mechanism.

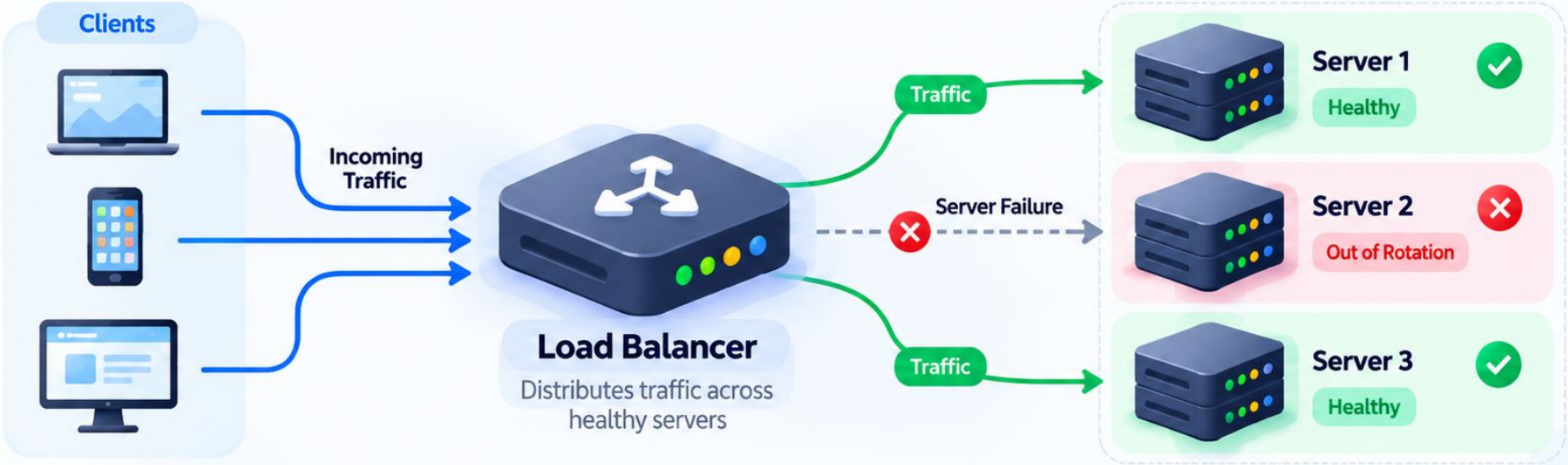


Introducing Load Balancing

1. A **load balancer** becomes the system's entry point.
2. Incoming requests are **distributed** across servers.
3. Traffic is no longer dependent on a **single machine**.
4. Applications can **scale more effectively**.



Load Balancing Enables Reliable Systems



Redundancy
Multiple servers provide redundancy.

Automatic Failover
Failed servers can be bypassed.

Safer Maintenance
Maintenance becomes safer.

Higher Availability & Resilience
Availability and resilience improve.

Instructor: Rahul Rajat Singh

Interview Questions on Load Balancing

1 Fundamentals

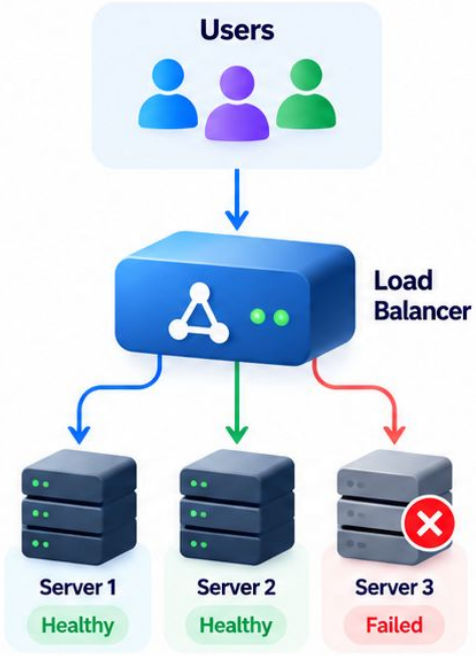
1. What problem does load balancing solve?
2. Why does a single server eventually become a bottleneck?
3. What role does a load balancer play in a distributed system?

2 Scaling & Architecture

4. What is the difference between vertical scaling and horizontal scaling?
5. Why is horizontal scaling often preferred over vertical scaling?
6. What challenge arises when an application is deployed across multiple servers?

3 Reliability & Availability

7. How does load balancing improve scalability?
8. How does load balancing improve availability?
9. What happens if one server fails in a load-balanced architecture?



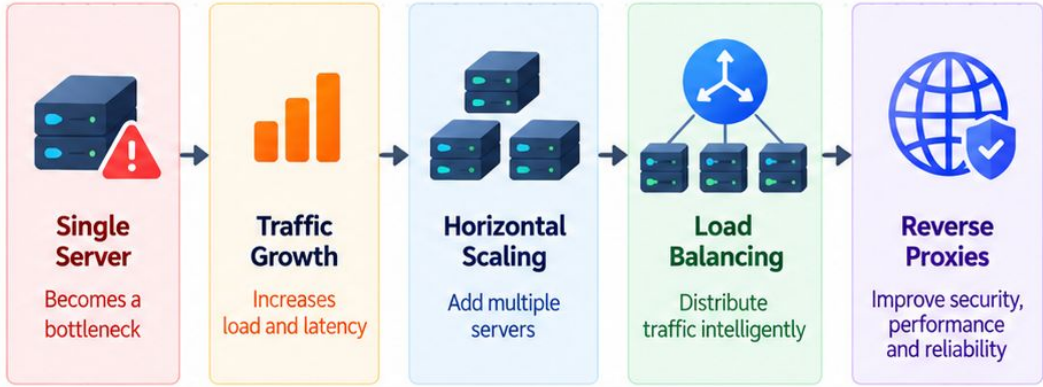
4 Design Thinking

10. Why is load balancing considered a foundational component of modern system design?
11. When should you introduce a load balancer into a system?
12. Walk me through the evolution from a single-server application to a load-balanced architecture.

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1. Single servers eventually become **scalability bottlenecks**.
2. Vertical scaling provides only **temporary relief**.
3. Horizontal scaling adds capacity through **multiple servers**.
4. Multiple servers require intelligent **traffic distribution**.
5. Load balancing enables **scalable** and **reliable systems**.



What's next:

- API Gateway

What is an API Gateway?

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Introduction to API Gateways

1. 

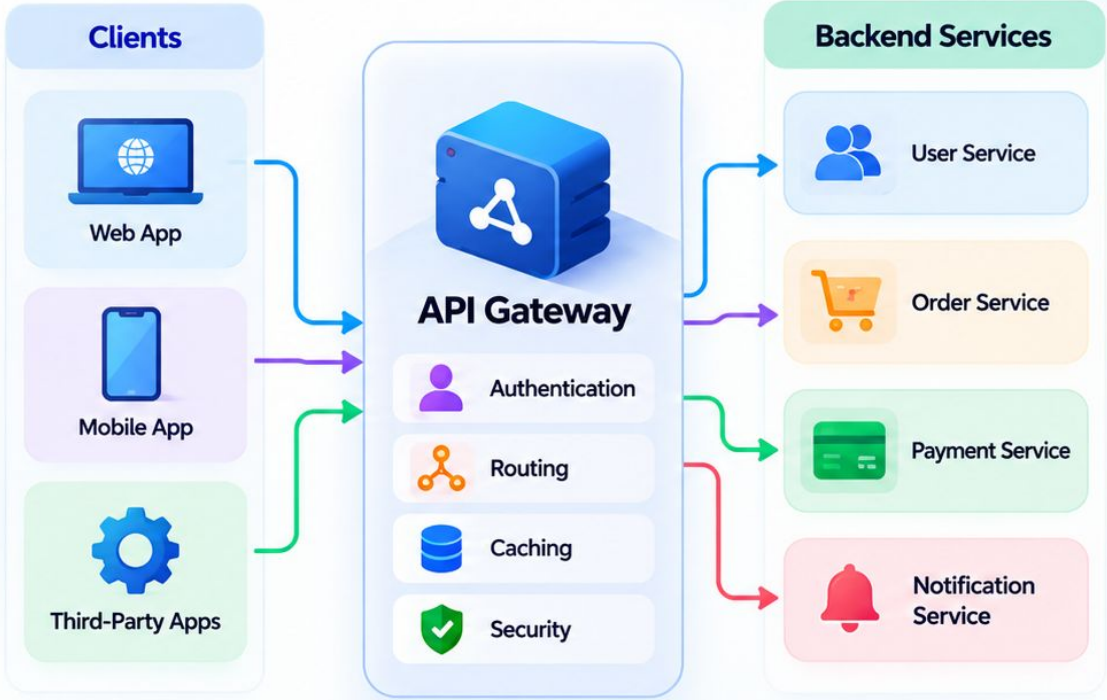
In modern system architecture, APIs connect **web, mobile, and third-party clients** to backend services.

2. 

Directly exposing backend services can lead to **performance, security, and management challenges**.

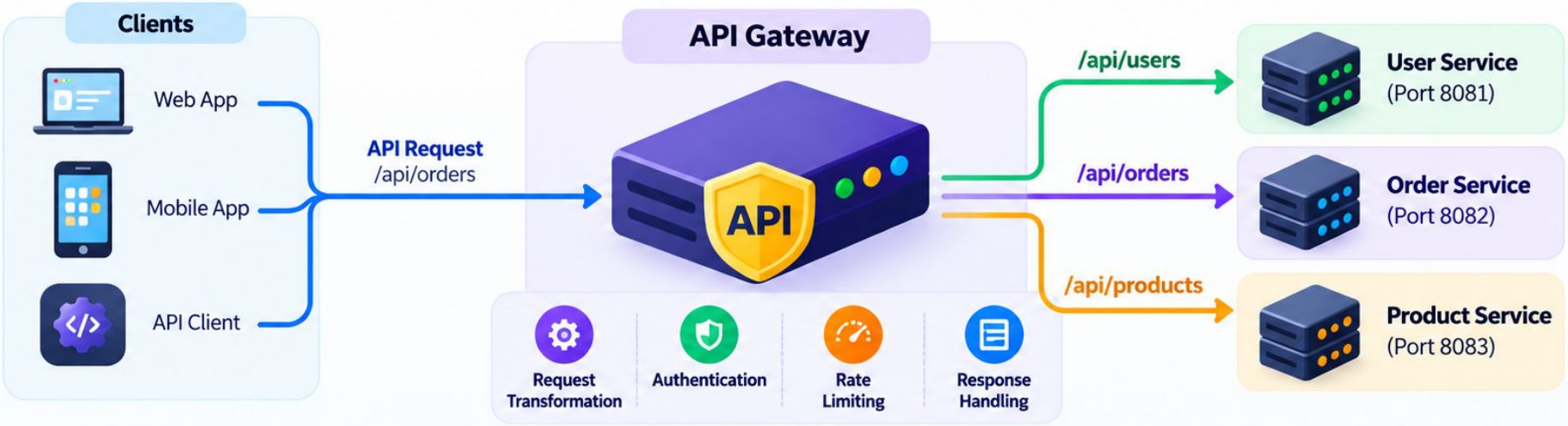
3. 

An API Gateway is a centralized entry point for all API requests, handling **authentication, routing, caching, and security**.



How API Gateways Work?

- API Gateway acts as a **reverse proxy** between clients and backend services.
- It receives API requests, processes them, and forwards them to the correct service.
- Key functions include request transformation, authentication, rate limiting, and response handling.



Why Use an API Gateway?

- Single entry point for all APIs
- Centralized security and authentication
- Apply rate limiting and throttling
- Simplifies client access to multiple services

Instructor: Rahul Rajat Singh

Benefits of Using an API Gateway

Security
Protects backend services from direct exposure.

Rate Limiting
Prevents abuse & DDoS attacks.

Load Balancing
Distributes traffic efficiently.

Caching
Speeds up response times.

Request Transformation
Converts request formats, protocols, and headers.

Logging & Monitoring
Tracks API performance and security events.



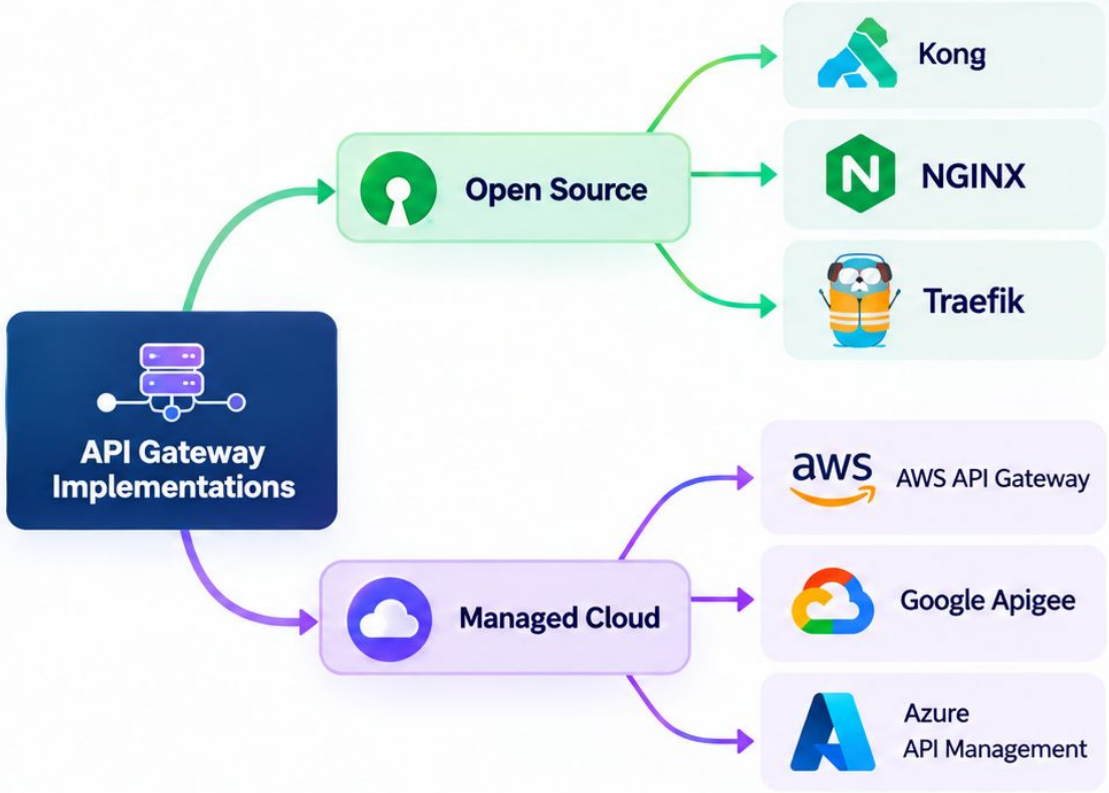
Popular API Gateway Implementations

Open-source solutions

- Kong
- NGINX
- Traefik

Cloud-based solutions

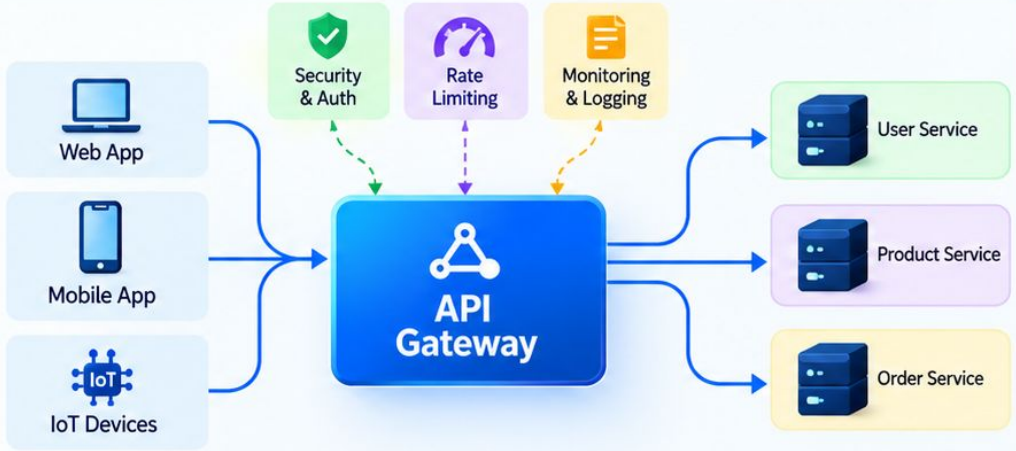
- AWS API Gateway
- Google Apigee
- Azure API Management



When to Use an API Gateway?

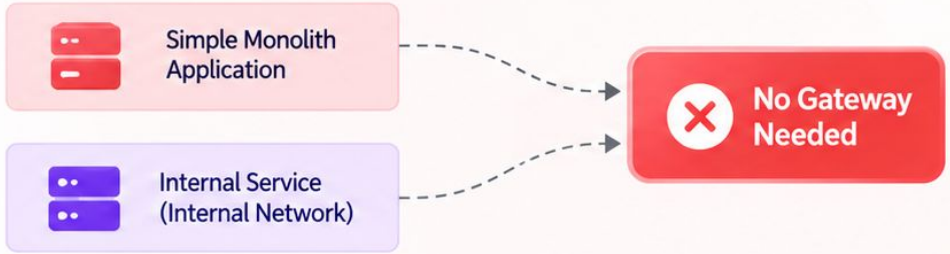
Best suited for:

- Microservices architectures.
- Multi-client APIs (Web, Mobile, IoT).
- APIs requiring security, rate limiting, and monitoring.



Avoid if:

- Simple monolithic apps with minimal API exposure.
- Low-traffic, internal-only services.



API Gateway – Interview Questions

1 Basic Questions

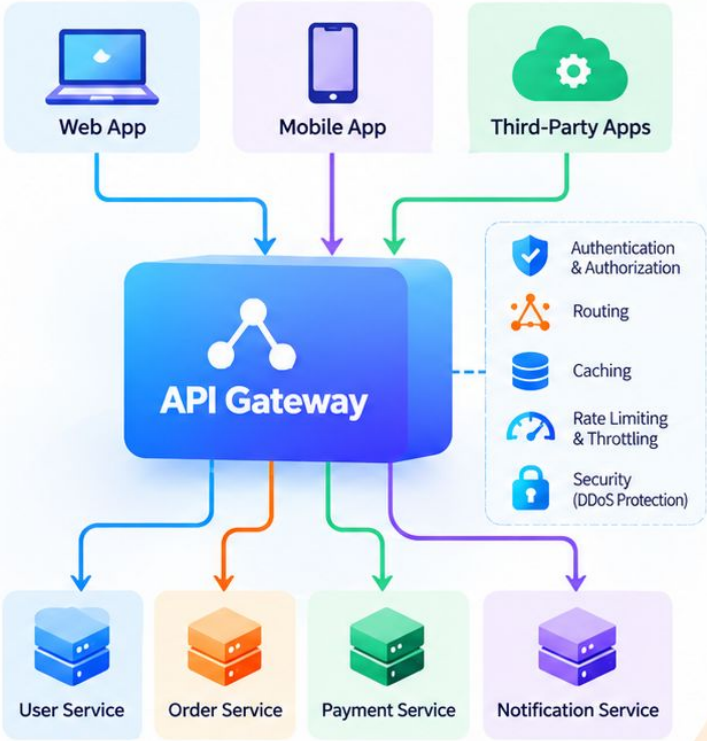
1. What is an API Gateway, and why is it used?
2. How does an API Gateway differ from a Load Balancer?
3. What are the key benefits of using an API Gateway?

2 Technical Questions

1. How does an API Gateway handle authentication and authorization?
2. Explain rate limiting and throttling in API Gateways.
3. What caching strategies can be implemented in an API Gateway?
4. How does an API Gateway improve security against DDoS attacks?

3 Scenario-Based Questions

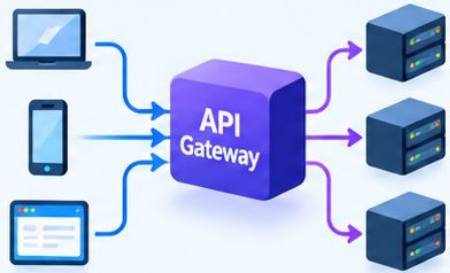
1. When should you use an API Gateway in a microservices architecture?
2. How would you design an API Gateway for a large-scale system with millions of users?
3. What challenges might arise when implementing an API Gateway, and how would you address them?



Conclusion & Key Takeaways

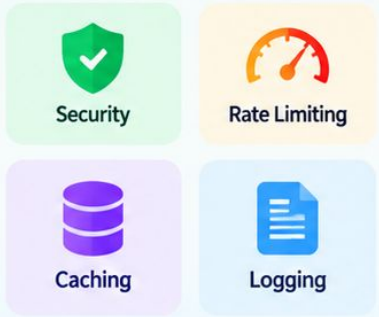
1 Centralized Entry Point

API Gateway is a centralized entry point for managing API traffic.



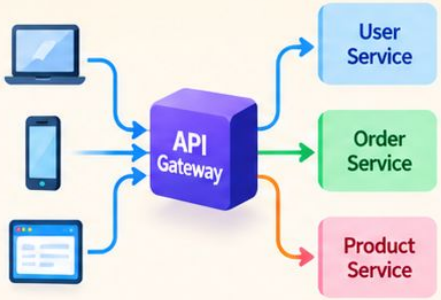
2 Security, Rate Limiting, Caching & Logging

Provides security, rate limiting, caching, and logging.



3 Useful for Modern Architectures

Useful for microservices & high-traffic applications.



4 Choose Based on Use Case & Scale

Choosing the right API Gateway depends on use case & scale.



What's Next?

Content Delivery Networks (CDNs)

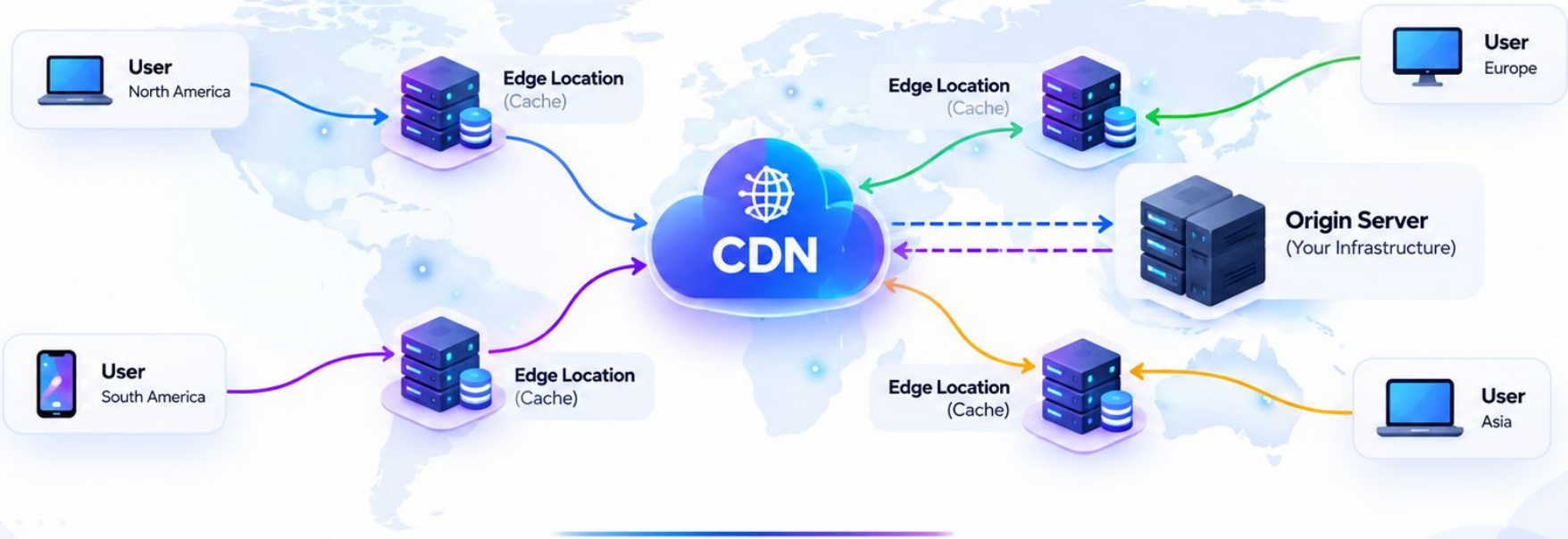


Learn how CDNs work, and how they help deliver content faster to users around the world.

Content Delivery Networks (CDN) in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

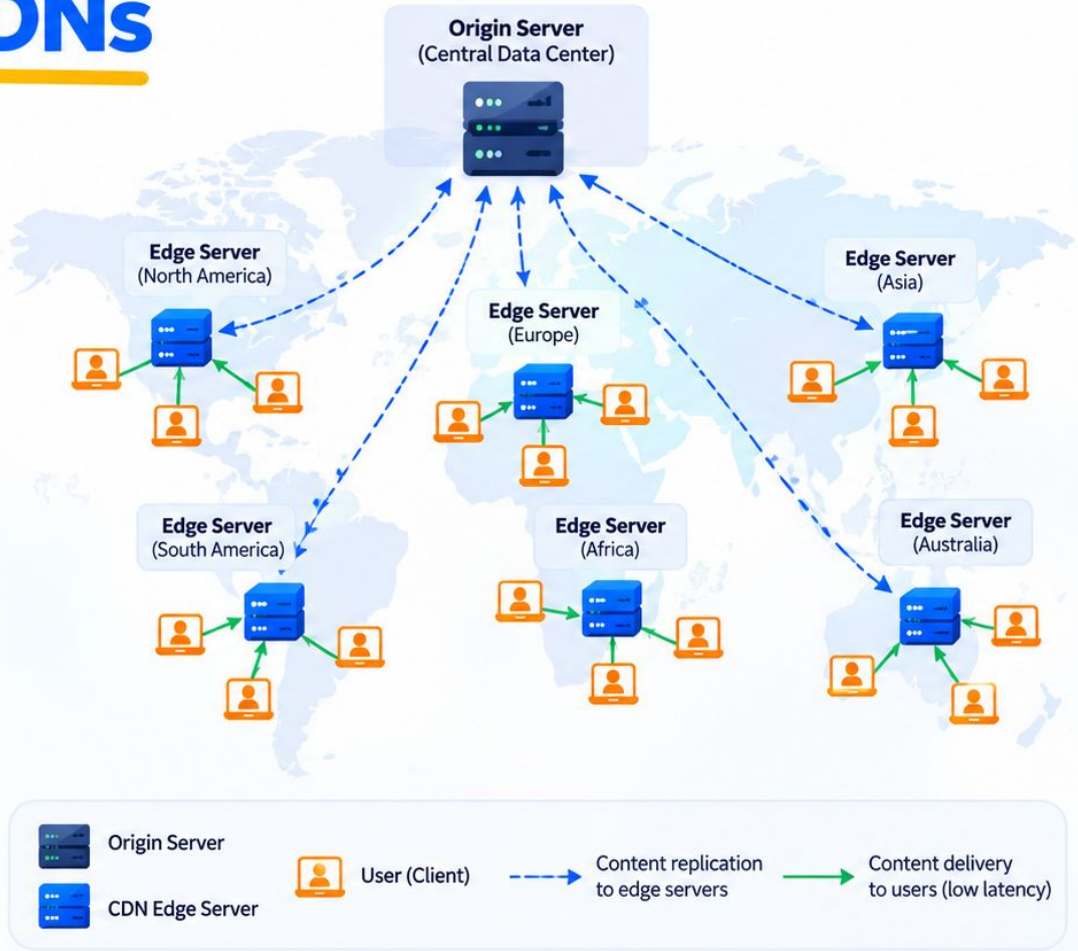
Introduction to CDNs

Definition:

A CDN is a globally distributed network of servers that work together to deliver content efficiently.

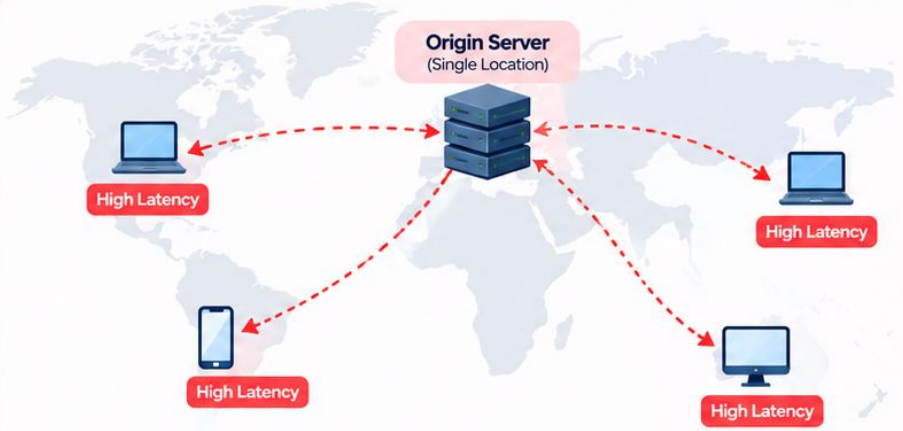
Why CDNs exist:

- Reduces **latency**
- Enhances **content availability**
- Improves **load handling**
- **Security**



Why are CDNs needed?

The problem without CDNs



High latency due to geographic distance

Users far from the origin server experience slower response times.

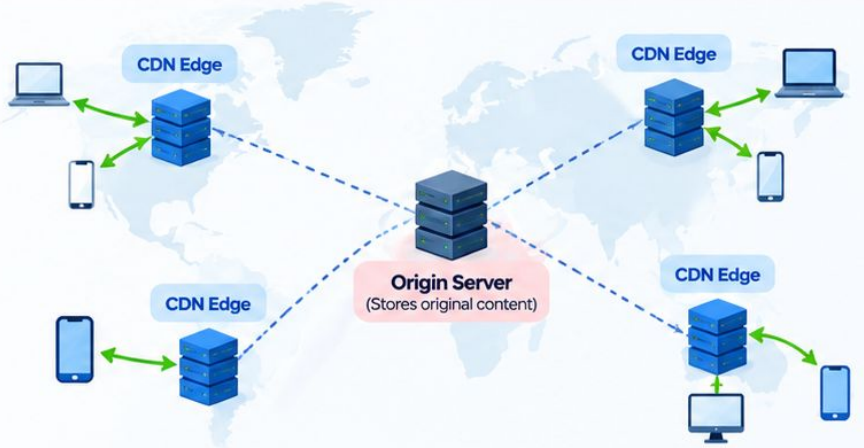
Overloaded origin servers

All traffic hits the same server, which can become a bottleneck during high demand.

Bandwidth constraints & slow load times

Large files and media consume more bandwidth, leading to slower downloads.

With CDNs (Content Delivery Networks)



Faster load times

Content is served from edge locations closer to users, reducing latency.

Less load on origin servers

Static content is cached at edge locations, reducing direct traffic to the origin server.

Better bandwidth utilization

Content is delivered efficiently from edges, reducing bandwidth constraints and improving overall performance.

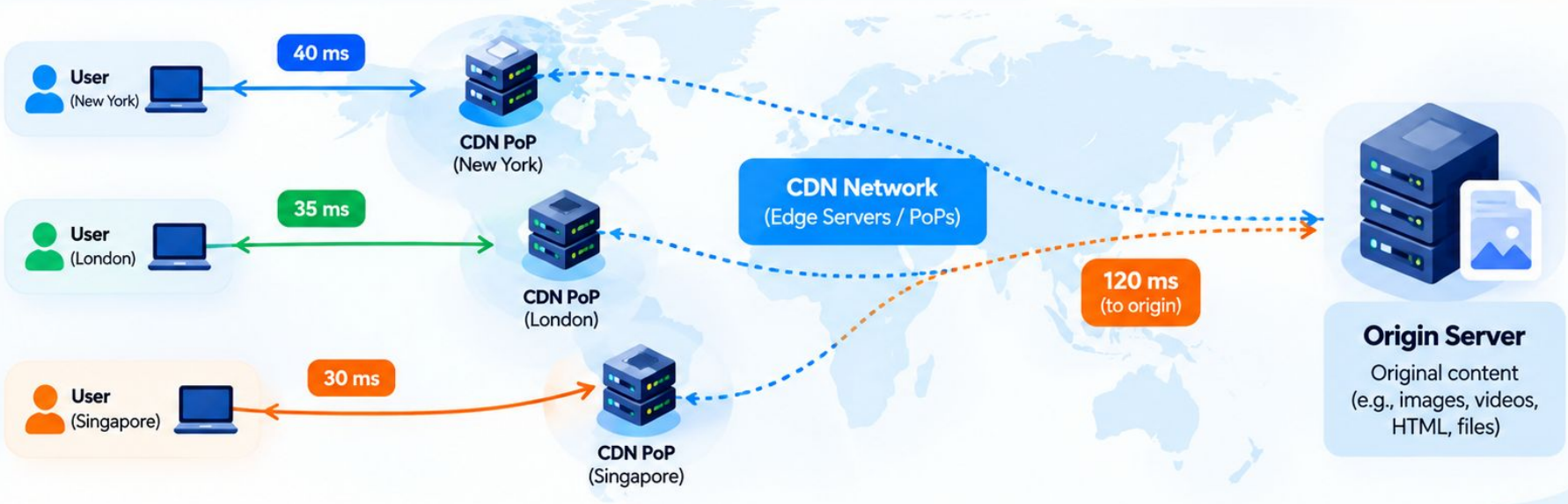
Instructor: Rahul Rajat Singh

CDN Architecture Overview

Origin Server
Stores the original content (e.g., images, videos, HTML, files).

Edge Servers / PoPs
Store cached content closer to users.

Request Routing
Directs users to the nearest PoP based on location and network conditions.



How a CDN Works and Handles a User Request

1 User requests content

User requests content (e.g., website, video, API data).

2 CDN routes to nearest edge

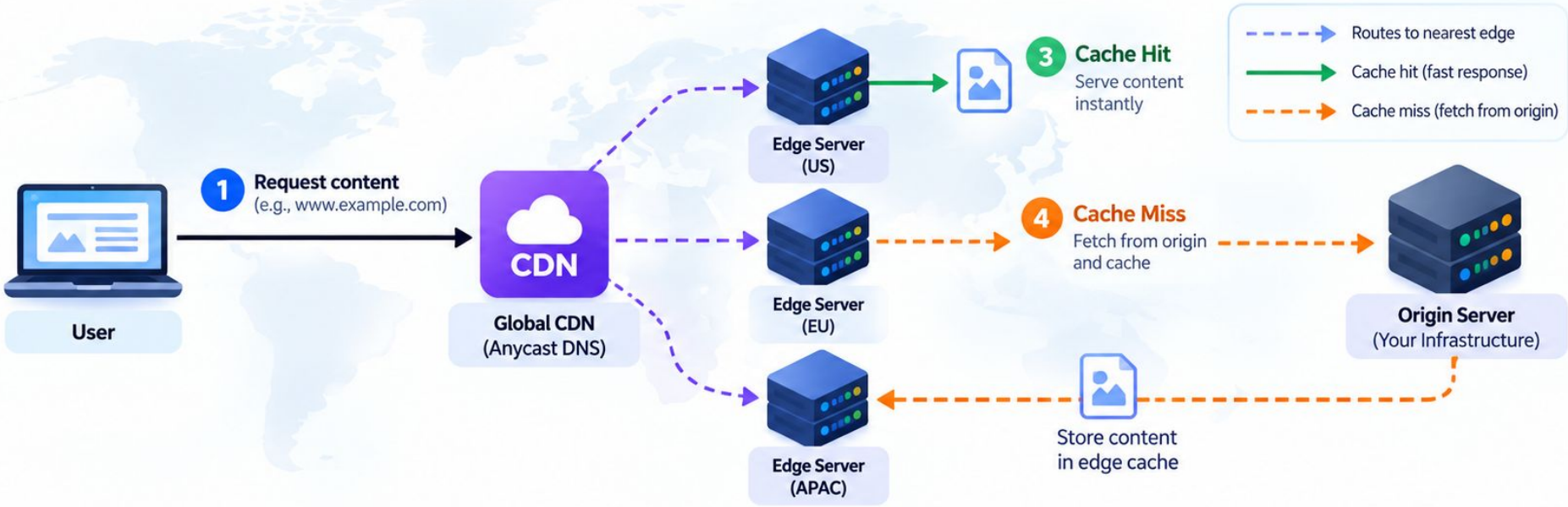
CDN directs the request to the nearest edge server based on geographic location, network latency, and server load.

3 Cache Hit

If the content is available on the edge server, it is delivered instantly.

4 Cache Miss

If the content is not cached, the request is forwarded to the origin server, fetched, and stored at the edge server for future use.



How CDNs Improve Performance, Reliability & Security

1 Caching & Replication
(Reduced Latency)

- Popular content is cached at edge locations worldwide.
- Cache TTL and invalidation strategies maintain content freshness.
- Reduces origin server load and accelerates content delivery.

2 Load Balancing & Intelligent Routing
(Improved Availability)

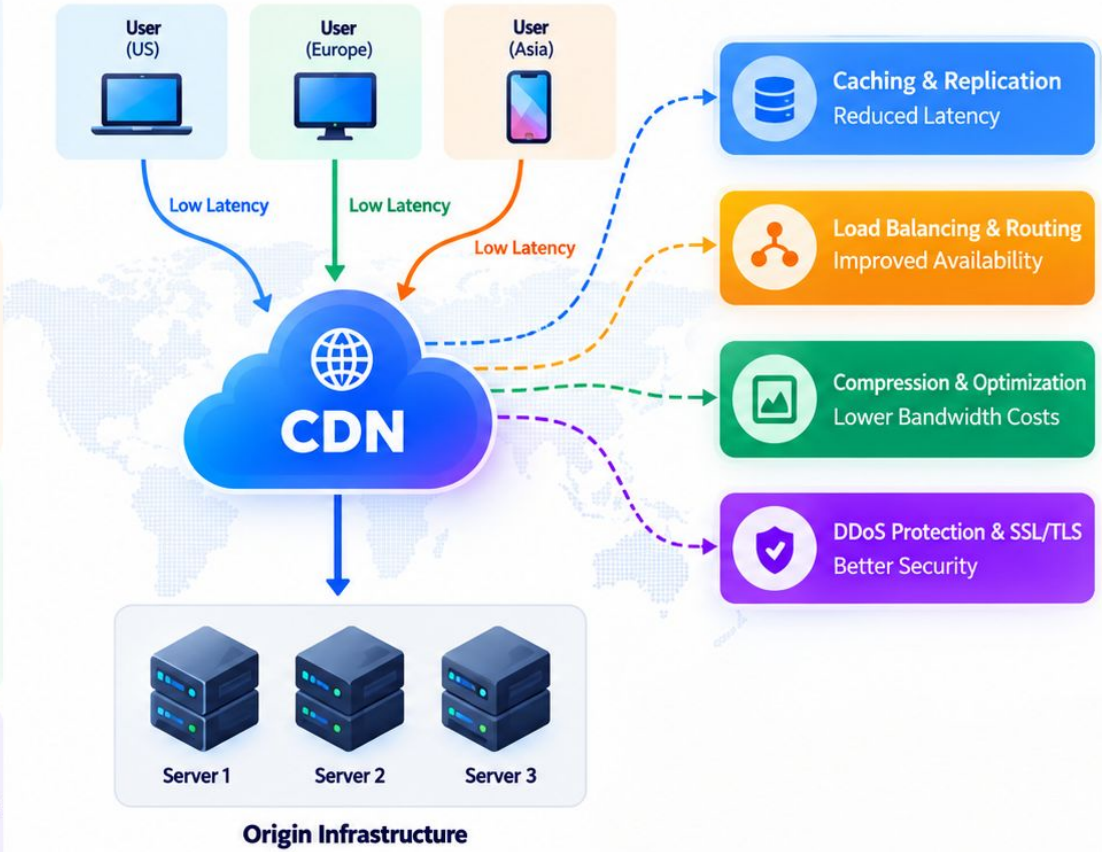
- Traffic is distributed across multiple PoPs.
- Geo-based, latency-based, and load-aware routing select the best edge location.
- Automatic failover reroutes traffic during outages.

3 Compression & Content Optimization
(Lower Bandwidth Costs)

- Gzip/Brotli compression reduces payload size.
- Image optimization (WebP, AVIF) improves delivery efficiency.
- CSS/JS minification reduces download times.

4 Security & Edge Protection
(Better Security)

- DDoS mitigation through rate limiting and traffic filtering.
- SSL/TLS termination secures communication.
- Protects origin infrastructure from direct exposure.



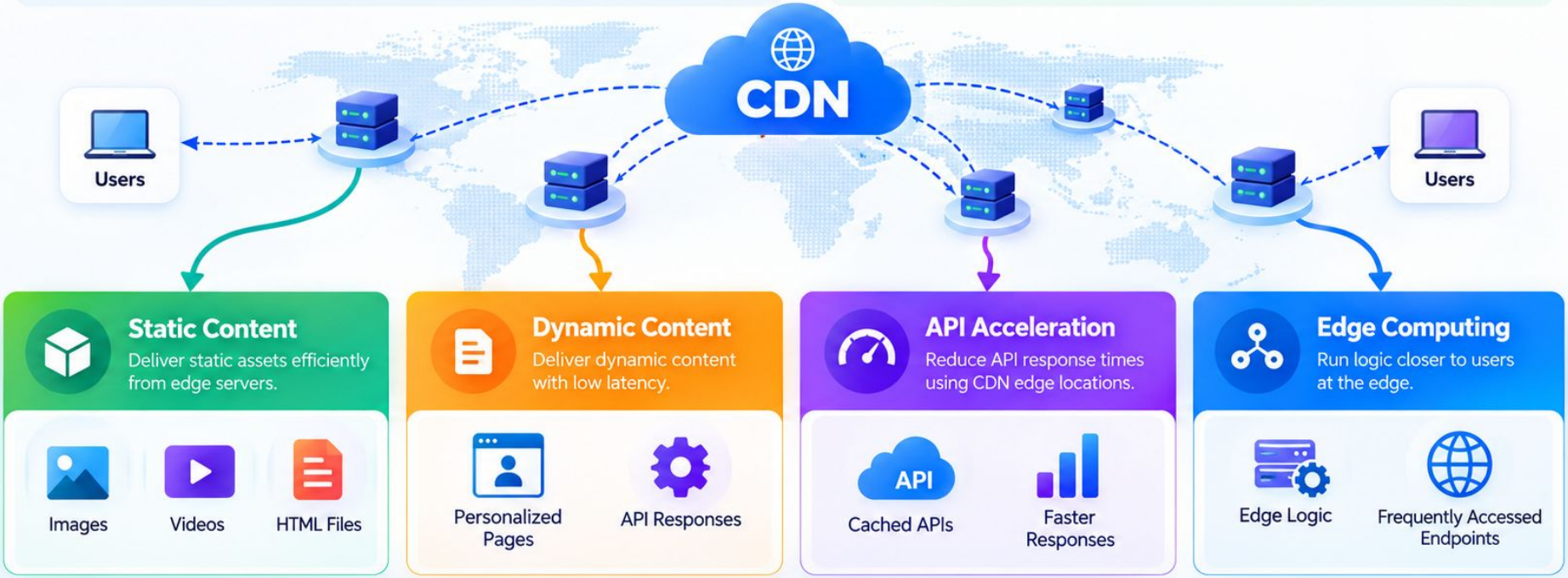
Use Cases of CDNs

Static vs. Dynamic Content Delivery

- **Static content:** Images, videos, HTML files.
- **Dynamic content:** API responses, personalized pages.

API Acceleration & Edge Computing

- Reduce API response times.
- Edge caching for frequently accessed endpoints.



Interview Questions - CDN



1 Basics of CDN

- What is a CDN, and how does it work?
- Why do we need CDNs in system design?
- What are the key benefits of using a CDN?



2 CDN Architecture & Components

- Explain the difference between an origin server and an edge server in a CDN.
- What is a PoP (Point of Presence) in a CDN?
- How does request routing work in a CDN?



3 Caching Strategies & Content Delivery

- What is a cache hit vs. cache miss, and how does a CDN handle them?
- Explain cache expiration and TTL (Time-To-Live) in a CDN.
- What are cache invalidation strategies, and why are they important?
- How does a CDN handle dynamic content caching?



4 Load Balancing & Failover Handling

- How do CDNs use load balancing to improve reliability?
- Explain different request routing strategies (Geo-based, Latency-based, Load-aware).
- What happens if a CDN PoP fails? How does failover handling work?



5 CDN Optimization Techniques

- What compression and minification techniques do CDNs use?
- How does a CDN optimize image and video delivery?
- How does API acceleration work in a CDN?



6 CDN Security & Challenges

- How does a CDN protect against DDoS attacks?
- What is SSL/TLS offloading, and why is it useful?
- What are some challenges in CDN implementation, and how can they be mitigated?



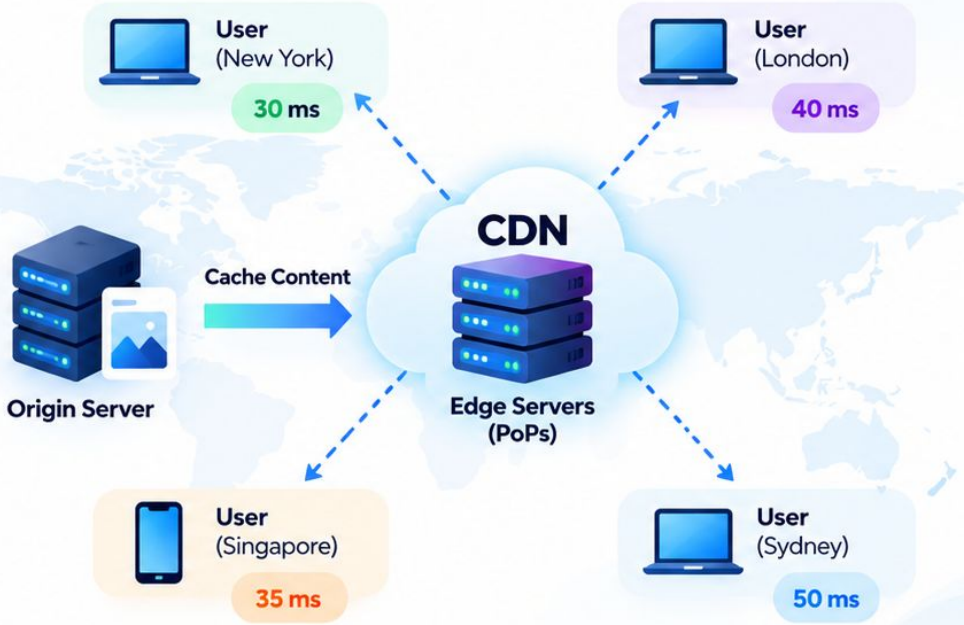
7 Advanced CDN Topics

- How does a multi-CDN architecture work?
- What is edge computing, and how does it relate to CDNs?
- How would you design a CDN for a large-scale video streaming platform?
- How do CDNs help in real-time applications like online gaming or stock trading?



Wrap-Up & Key Takeaways

1. CDNs improve performance by **caching and distributing content** globally.
2. They **reduce latency**, improve load times, and lower bandwidth costs.
3. CDNs enhance security with **DDoS protection, SSL/TLS encryption**, and WAFs.
4. Use cases include **static content delivery, API acceleration**, and **edge computing**.



Next Topic:

Networking Fundamentals

Summary & Best Practices

Summary of Covered Topics – Networking & Communication

1. **Introduction to Networking in System Design**
   Why networking is crucial in distributed systems.
2. **Understanding IP Addresses**
   Differences between IPv4 & IPv6, public vs. private IPs.
3. **How DNS Works**
   Domain Name Resolution, caching, and its role in large-scale systems.
4. **Client-Server Model Explained**
   How clients and servers interact using request-response cycles.
5. **Forward Proxy vs. Reverse Proxy**
   Their differences, use cases, and impact on system design.
6. **Introduction to Load Balancing**
   Introduction and why it's needed.
7. **What is an API Gateway?**
   Managing API traffic, enforcing security, and enabling efficient routing.
8. **Content Delivery Networks (CDN) in System Design**
   How CDNs improve performance and reduce latency.



Next Topic:
Protocols